



BIG DATA ANALYSIS

BICTE 8TH SEMESTER | COMPLETE NOTEBOOK

✓ DATA VISUALIZATION ✓ STATISTICAL ANALYSIS ✓ PREDICTIVE MODELING



ajaymahato@notedinsights.com

AJAY MAHATO

www.notedinsights.com



Table of Contents

Chapter 1: Concept of Big Data	1
1.1 Definition and Characteristics (Volume, Velocity, Variety, Veracity, Value)	1
1.2 Applications of Big Data in Education System	3
1.3 Big Data vs. Traditional Data.....	4
1.4 Recent Trends in Big Data and Practices in Education system Recent Trends in Big Data 7	
Chapter 2: Basic Concepts of Hadoop	10
2.1 Hadoop Ecosystem Overview	10
2.2 Hadoop Distributed File System (HDFS)	11
2.3 MapReduce Framework.....	15
2.4 Hadoop YARN	17
2.5 Concept of Hadoop Applications: Hive, Spark, Sqoop, Pig	19
Chapter 3: Data Visualization with Power BI.....	32
3.1 Power BI Overview and Components.....	32
3.2 Connecting to Data Sources.....	34
3.3 Data Transformation and Modeling.....	34
3.4 Creating Visualizations	38
3.5 Sharing and Collaborating with Power BI	38
Chapter 4: Data Preprocessing for Big Data.....	42
4.1 Data Cleaning and Transformation	42
4.2 Handling Missing and Inconsistent Data	44
4.3 Introduction to ETL (Extract, Transform, Load) Processes	47
4.4 Concept of Data Integration and Aggregation	50
4.5 Data Reduction and Feature Selection.....	54
Chapter 5: Data Analysis Using Python Libraries	57
5.1 Introduction to Data Analysis and Visualization.....	57
5.2 Concept of python library and bigdata analysis environment	61
5.3 NumPy and Statistical functions.....	63
5.4 Pandas for Data Handling	66

5.5	Matplotlib for Data Visualization	70
5.6	Data Ingestion and Exploration in python	73
Unit 6: Big Data Analysis Using Python Libraries		79
6.1	Introduction to Big Data and PySpark	79
6.2	Installing and configuring PySpark	80
6.3	PySpark DataFrames.....	83
6.4	Big Data Analysis with PySpark.....	86
6.5	Using SQL (DBMS tools) for querying data	88
6.6	Data Visualization and Reporting	90
6.7	Concept of Spark SQL and Its Application.....	92

Chapter 1: Concept of Big Data

1.1 Definition and Characteristics (Volume, Velocity, Variety, Veracity, Value)

Big data refers to extremely large and diverse collections of structured, unstructured, and semi-structured data that continues to grow exponentially over time. These datasets are so huge and complex in volume, velocity, and variety, that traditional data management systems cannot store, process, and analyze them.

The amount and availability of data is growing rapidly, spurred on by digital technology advancements, such as connectivity, mobility, the Internet of Things (IoT), and artificial intelligence (AI). As data continues to expand and proliferate, new big data tools are emerging to help companies collect, process, and analyze data at the speed needed to gain the most value from it.

Big data describes large and diverse datasets that are huge in volume and also rapidly grow in size over time. Big data is used in machine learning, predictive modeling, and other advanced analytics to solve business problems and make informed decisions.

Here are some big data examples that are helping transform organizations across every industry:

- Tracking consumer behavior and shopping habits to deliver hyper-personalized retail product recommendations tailored to individual customers
- Monitoring payment patterns and analyzing them against historical customer activity to detect fraud in real time
- Combining data and information from every stage of an order's shipment journey with hyperlocal traffic insights to help fleet operators optimize last-mile delivery
- Using AI-powered technologies like natural language processing to analyze unstructured medical data (such as research reports, clinical notes, and lab results) to gain new insights for improved treatment development and enhanced patient care
- Using image data from cameras and sensors, as well as GPS data, to detect potholes and improve road maintenance in cities
- Analyzing public datasets of satellite imagery and geospatial datasets to visualize, monitor, measure, and predict the social and environmental impacts of supply chain operations

The 3 Vs of Big Data (Characteristics of Big Data)

Big data definitions may vary slightly, but it will always be described in terms of volume, velocity, and variety. These big data characteristics are often referred to as the “3 Vs of big data” and were first defined by Gartner in 2001.

1. Volume

As its name suggests, the most common characteristic associated with big data is its **high volume**. This describes the enormous amount of data that is available for collection and produced from a variety of sources and devices on a continuous basis.

2. Velocity

Big data velocity refers to the **speed at which data is generated**. Today, data is often produced in real time or near real time, and therefore, it must also be processed, accessed, and analyzed at the same rate to have any meaningful impact.

3. Variety

Data is **heterogeneous**, meaning it can come from many different sources and can be **structured, unstructured, or semi-structured**. More traditional structured data (such as data in spreadsheets or relational databases) is now supplemented by unstructured text, images, audio, video files, or semi-structured formats like sensor data that can't be organized in a fixed data schema.

In addition to these three original Vs, three others that are often mentioned in relation to harnessing the power of big data: veracity, variability, and value.

4. Veracity (Accuracy)

Big data isn't always clean. It can be messy, full of noise, and prone to errors—which makes it hard to trust unless properly filtered and verified. Huge datasets can feel overwhelming, and small ones might leave out important details. The bottom line: the more accurate and reliable your data is, the more you can trust the insights it gives you.

5. Variability (Inconsistency)

Data doesn't always mean the same thing over time. Its meaning can shift based on context, interpretation, or even how it was collected. Different methods and changing goals can lead to inconsistency, making analysis tricky unless you're keeping track of those changes.

6. Value

Not all data is useful. What really matters is whether it brings actual value to your business. It's not just about collecting tons of information—it's about collecting the *right* information and analyzing it in a way that leads to smart decisions and real results.

1.2 Applications of Big Data in Education System

1. Smarter Evaluation

Traditionally, student performance was judged only by exam scores. Big Data changes that. It pulls information from a range of sources—not just tests—to give a more complete and fair evaluation of a student's abilities.

With advanced analytics tools, schools can now provide better grading systems, reward participation in new programs, and help students understand how they're being assessed. It's about making grading more transparent and meaningful.

2. Tracking Progress and Performance

Educational institutions handle a massive amount of data daily. Big Data helps streamline operations and gives a clearer picture of student accomplishments. It provides deep insights into how students are engaging and performing. For example, it can track:

- How quickly students respond to questions
- Which questions they skip
- What extra resources they use
- Which answers they get right

All this data can be used to improve online learning, boost motivation, and better reward student performance beyond just letter grades.

3. Personalized Learning

Every student learns differently. Big Data helps identify those differences and tailors the learning experience to fit each student's needs.

It enables:

- Skill-based learning paths
- Personalized results and feedback
- Improved communication between students and teachers

With the help of smart tech and analytics, students get a more engaging and custom experience—something that actually adapts to how they learn best.

4. Better Educational Systems

Traditional grading systems have been criticized for being too rigid and outdated. Big Data can change that by giving teachers and institutions tools to monitor both student performance and behavior more effectively.

It can generate regular reports, offer detailed analytics, and help educators focus on the areas where students need the most support.

5. Reducing Dropout Rates

Dropouts are a serious concern in many institutions. Big Data can help spot warning signs early by analyzing:

- Attendance patterns
- Grades over time
- Behavioral changes

By catching these red flags early, schools can take steps to keep students on track and prevent dropouts before they happen.

6. Streamlined University Admissions

Admissions can be chaotic. During peak times, schools have to process thousands of student applications and tons of data. Big Data can make this smoother by analyzing application trends, predicting enrollment numbers, and helping universities make more informed decisions during admissions.

The Bigger Picture: Why Big Data Matters in Education

Big Data isn't new—but its role in education is growing fast. It helps uncover gaps in learning, adapt to students' individual preferences, and support both traditional and remote learning models. Students benefit from more relevant and personalized experiences, teachers get better tools to guide them, and institutions can make smarter decisions based on actual data—not just assumptions.

At the end of the day, Big Data has the power to transform education—not just in schools, but across colleges and universities too. It can improve how we teach, how we learn, and how we grow.

1.3 Big Data vs. Traditional Data

Data is information that helps businesses and organizations make decisions. Based on volume, variety, velocity, and mode of handling data, traditional data, and big data. It is quite helpful for organizations to understand these key dissimilarities to enable them to select the right approach in data storage, data processing, and data analysis.

Traditional data is the kind of information that is easy to organize and store in simple databases, like spreadsheets or small computer systems. This could be things like customer names, phone numbers, or sales records.

Big data, however, is much larger and more complex. It includes huge amounts of information from many different sources, such as social media, online videos, sensors in machines, or website clicks. Big data is harder to organize because it's so large and comes in different forms, making it

difficult for traditional tools to handle. In this article, we are going to discuss the different between traditional data and big data in detail.

1.3.1 What is Traditional Data?

Traditional data is the structured data that is being majorly maintained by all types of businesses starting from very small to big organizations. In a traditional database system, a centralized database architecture is used to store and maintain the data in a fixed format or fields in a file. For managing and accessing the data Structured Query Language (SQL) is used.

Traditional data is characterized by its high level of organization and structure, which makes it easy to store, manage, and analyze. Traditional data analysis techniques involve using statistical methods and visualizations to identify patterns and trends in the data.

Traditional data is often collected and managed by enterprise resource planning (ERP) systems and other enterprise-level applications. This data is critical for businesses to make informed decisions and drive performance improvements.

1.3.2 Advantages of Traditional Data

- Stored, managed easier with regular database systems.
- Great efficiency in data access and manipulation arises from having a structured data model.
- It is relatively inexpensive in storing and processing of data or datasets that are relatively small in size.

1.3.3 Disadvantages of Traditional Data

- This tends to be only applicable in structured formats and can therefore limit flexibility.
- This does not work well for unstructured data types such as text, images or videos.
- Worse when the number of inputs becomes too large and hence difficult to scale.

1.3.4 What is Big Data?

We can consider big data an upper version of traditional data. Big data deal with too large or complex data sets which is difficult to manage in traditional data-processing application software. It deals with large volume of both structured, semi structured and unstructured data. Volume, Velocity and Variety, Veracity and Value refer to the 5'V characteristics of big data. Big data not only refers to large amount of data it refers to extracting meaningful data by analyzing the huge amount of complex data sets.

Big data is characterized by the three Vs: volume, velocity, and variety. Volume refers to the vast amount of data that is generated and collected; velocity refers to the speed at which data is generated and must be processed; and variety refers to the many different types and formats of data that must be analyzed, including structured, semi-structured, and unstructured data.

Due to the size and complexity of big data sets, traditional data management tools and techniques are often inadequate for processing and analyzing the data. Big data technologies, such as Hadoop, Spark, and NoSQL databases, have emerged to help organizations store, manage, and analyze large volumes of data.

1.3.5 Advantages of Big Data

- It covers structured, semi structured as well as un structured data.
- Allows use of sophisticated analyses such as forecasting and artificial intelligence.
- This is scalable meaning organizations can handle huge volumes of data in the system.

1.3.6 Disadvantages of Big Data

- Physical infrastructure needed reduces adaptability and so the cost is generally higher.
- The analysis of big data takes time.
- Protecting data privacy and security is even harder.

1.3.7 Some Similarities Between Traditional Data and Big Data

1. Data Quality

The quality of data is essential in both traditional and big data environments. Accurate and reliable data is necessary for making informed business decisions.

2. Data Analysis

Both traditional and big data require some form of analysis to derive insights and knowledge from the data. Traditional data analysis methods typically involve statistical techniques and visualizations, while big data analysis may require machine learning and other advanced techniques.

3. Data Storage

In both traditional and big data environments, data needs to be stored and managed effectively. Traditional data is typically stored in relational databases, while big data may require specialized technologies such as Hadoop, NoSQL, or cloud-based storage systems.

4. Data Security

Data security is a critical consideration in both traditional and big data environments. Protecting sensitive information from unauthorized access, theft, or misuse is essential in both contexts.

5. Business Value

Both traditional and big data can provide significant value to organizations. Traditional data can provide insights into historical trends and patterns, while big data can uncover new opportunities and help organizations make more informed decisions.

S.N.	Traditional Data	Big Data
1	Generated within organizations	Generated from multiple sources (social media, devices)
2	Small in size (GB to TB)	Very large in size (PB to ZB)
3	Structured data only	Structured, semi-structured, and unstructured data
4	Stored and managed centrally	Stored and managed in a distributed manner
5	Data grows slowly (per hour/day)	Data grows very fast (per second)
6	Easy to manage with normal systems	Hard to manage, needs powerful systems
7	Uses traditional tools like RDBMS	Uses advanced tools like Hadoop, Spark, NoSQL

1.4 Recent Trends in Big Data and Practices in Education system Recent Trends in Big Data

We're living in a time where technology is reshaping nearly every aspect of life—and education is no exception. Big Data, along with Artificial Intelligence (AI) and Machine Learning (ML), is driving major changes in how institutions teach, evaluate, and support students. After the COVID-19 pandemic, the push toward digital learning made the role of data even more critical in education—just like in healthcare or e-commerce.

Let's break down some of the key trends in Big Data and how they're making an impact on the education system:

1. Expanding IoT Networks

With the growth of 5G and the Internet of Things (IoT), we're seeing more connected devices than ever—collecting and sharing real-time data.

In education, this looks like:

- Smart campuses with sensors that track energy use, improve security, or monitor facility usage.
- Classrooms with IoT-enabled tools like digital whiteboards, automatic attendance systems, and real-time learning feedback.

2. AI Is Becoming Easier to Use

Thanks to low-code and no-code platforms, AI is no longer just for tech giants. Schools and universities can now use AI tools without needing expert-level developers.

Examples in education:

- AI-based learning apps that adapt to each student's needs.
- Chatbots helping students with admissions or course questions.
- Automated systems that handle grading and give instant feedback.

3. Predictive Analytics is on the Rise

Predictive analytics uses historical data to forecast what might happen next. It's all about being proactive, not just reactive.

Educational use cases:

- Spotting students who may be at risk of failing or dropping out.
- Planning enrollment and allocating resources based on past trends.
- Customizing course content based on how students typically perform.

4. Unlocking “Dark Data” with the Cloud

Most institutions sit on a goldmine of data they've never actually used—this is called “dark data.” Moving it to the cloud makes it accessible and usable.

How it helps education:

- Analyzing archived data like old student records to find long-term trends.
- Combining scattered data sources to improve student support.
- Using BI (Business Intelligence) tools for smarter decisions.

5. More Institutions Hiring Chief Data Officers (CDOs)

As data becomes more central to planning and strategy, many organizations are hiring CDOs to lead data efforts.

Their role in education:

- Setting strong data management and privacy policies.
- Ensuring departments use accurate, reliable data.
- Helping leaders base decisions on actual data, not guesswork.

6. The Potential of Quantum Computing

Still emerging, quantum computing promises to solve complex problems much faster than traditional computers.

Possible uses in education:

- Running advanced simulations for research in science or medicine.

- Making complicated systems like class scheduling more efficient.
- Modeling learning environments and behaviors with greater accuracy.

7. Stronger Cybersecurity with AI and ML

As schools generate more digital data, cybersecurity becomes a bigger priority. AI-powered security tools now help detect and prevent threats in real time.

Impact on education:

- Protecting sensitive student and staff data.
- Ensuring safe online learning platforms.
- Monitoring school networks for any unusual activity or breaches.

8. The Rise of Open-Source Data

Open-source tools and datasets are making research and innovation more accessible for everyone.

In practice:

- Using platforms like TensorFlow or Hadoop for academic projects.
- Giving students access to public datasets to build their data skills.
- Promoting a culture of collaboration and transparency in education.

9. Data Democratization

This is all about giving more people—not just IT teams—access to data. With user-friendly dashboards and tools, teachers and administrators can use data to make better decisions.

Real benefits include:

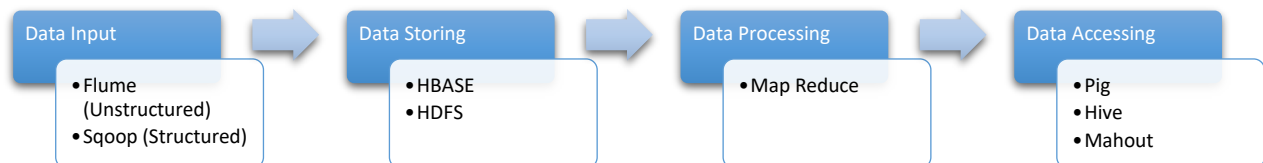
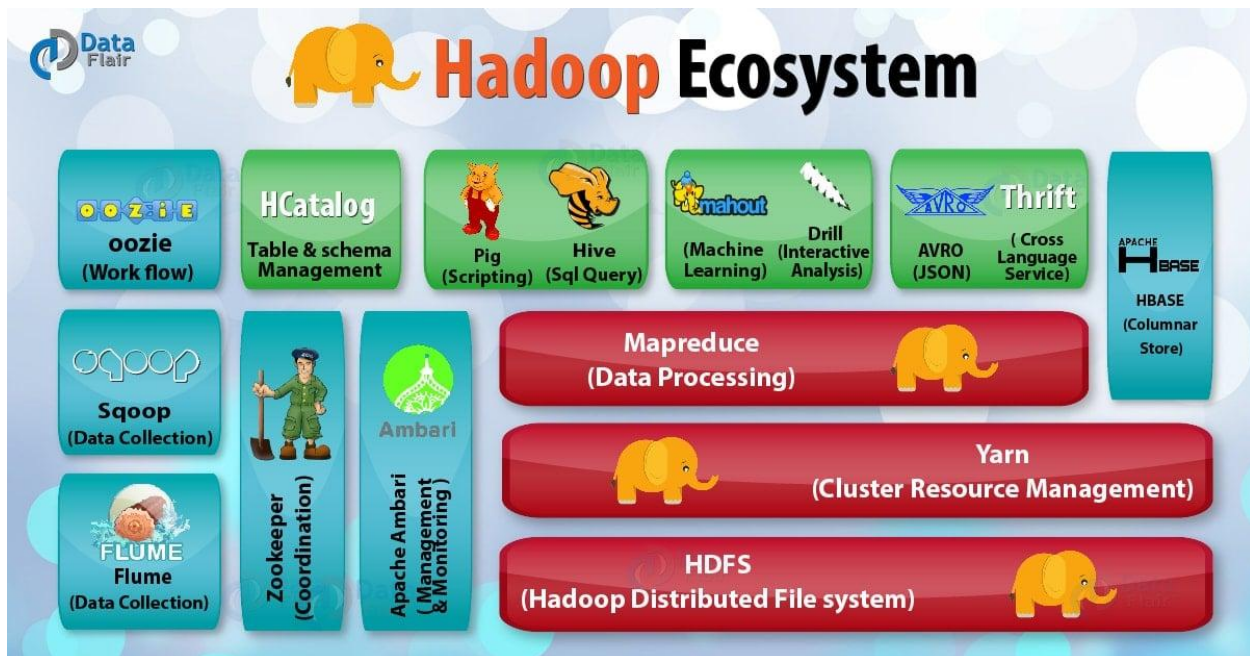
- Teachers using analytics to adapt lesson plans.
- Administrators involving more stakeholders in planning.
- Decisions being made faster, backed by real-time insights.

Chapter 2: Basic Concepts of Hadoop

2.1 Hadoop Ecosystem Overview

Hadoop is an open-source framework designed for processing and storing massive amounts of data in a distributed computing environment. It's a key player in the world of big data and provides the foundation for many large-scale data processing applications.

Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions.



2.1.1 Key Benefits of the Hadoop Ecosystem

1. It Helps You Store, Manage, And Process Data

The Hadoop ecosystem is a collection of open-source software tools that allow you to store and process large amounts of data. The tools in this ecosystem include HDFS (Hadoop Distributed File

System), YARN (Yet Another Resource Negotiator), and MapReduce. These tools allow you to store huge amounts of data on your servers without paying for expensive third-party solutions.

2. It Allows You to Run Analytics on Large Datasets

The Hadoop ecosystem allows you to run analytics on large datasets using MapReduce programming models. This approach lets you run complex algorithms on your data without purchasing expensive hardware or software solutions from third-party vendors.

3. It Enables Real-time Analysis of Streaming Data

The Hadoop ecosystem is designed to process large amounts of data in real time. This can be useful for many applications, such as financial transactions and tracking customer behavior.

4. It offers Security for Your Data

Hadoop offers some degree of security through encryption and authentication methods that protect your data from being accessed by unauthorized users or systems.

#Components of Hadoop – HDSS – Mapreduce – Yarn – Others

2.2 Hadoop Distributed File System (HDFS)

2.2.1 Introduction to HDFS

The Hadoop Distributed File System (HDFS) is the primary storage system used by Hadoop applications. It is a distributed file system designed to handle large datasets on commodity hardware, offering high fault tolerance, scalability, and throughput. HDFS is an open-source framework initially developed for the Apache Nutch web search engine and is now a subproject of Apache Hadoop (<https://hadoop.apache.org/hdfs/>). It supports big data analytics by providing reliable and efficient data storage and access across distributed clusters.

HDFS is optimized for high-throughput access to large datasets, relaxing some POSIX requirements to enable streaming data access. It is widely used in Hadoop ecosystems for managing pools of big data and supporting related analytics applications.

2.2.2 HDFS Architecture

HDFS operates on a **primary/secondary architecture**, consisting of a single **Name Node** (the primary node) and multiple **Data Nodes** (worker nodes). This architecture enables efficient management and storage of data across a scalable cluster.

Key Components

1. **Name Node (Master):**

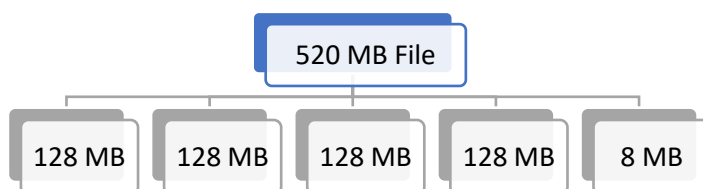
- The Name Node is the central controller of HDFS, managing the file system namespace and client access to files.
- **Functions:**
 - Performs file system operations such as opening, closing, renaming files, and directories.
 - Maps data blocks to Data Nodes and maintains metadata (e.g., file names, permissions, and block locations).
 - Stores namespace (meta data of files) to map which file stores in which data node.
 - Monitors Data Node health via periodic heartbeat signals and block reports.
 - Stores same data at multiple Data Nodes to prevent system failure.

2. Data Nodes (Worker):

- Data Nodes are worker nodes that store and manage data blocks on commodity hardware.
- **Functions:**
 - Serve read and write requests from clients.
 - Perform block creation, deletion, and replication as instructed by the Name Node.
 - Send periodic heartbeat signals to the Name Node to confirm their status.
 - Provide block reports to the Name Node to track stored blocks.
 - Store each block with a default replication factor of three (one original and two replicas) for redundancy.

2.2.3 File Storage in HDFS

- Files in HDFS are split into blocks (typically 128 MB or 256 MB) and distributed across Data Nodes.
- The file system namespace follows a hierarchical structure similar to traditional file systems, allowing users to create, remove, rename, or move files and directories.
- The replication factor ensures data redundancy, with each block stored on multiple Data Nodes.



Distributed over Data Nodes to access and process parallelly later.

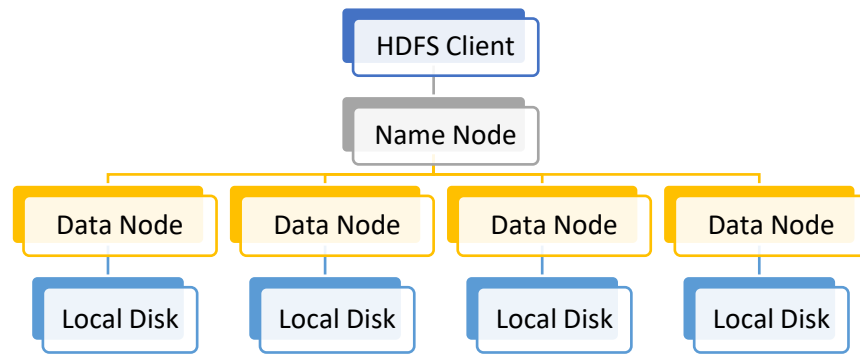


Figure: HDFS Storage System

2.2.4 Features of HDFS

HDFS is designed with several key features that make it suitable for big data applications:

1. Data Replication

HDFS automatically copies each block of data to multiple machines (called Data Nodes). This ensures that even if one machine fails, your data isn't lost—another copy is ready to go.

2. Fault Tolerance & Reliability

Because of this replication strategy, HDFS can handle failures smoothly. If a node crashes, the system keeps running and pulls the data from another location without interruption.

3. High Availability

Even if a Name Node or Data Node goes down, your data stays accessible. The system is designed to stay up and running as long as possible.

4. Scalability

Need more storage? Just add more machines. HDFS is built to grow with you, scaling across hundreds—or even thousands—of nodes.

5. High Throughput

By spreading data across nodes and processing it in parallel, HDFS enables fast, large-scale data operations. This makes it perfect for big data applications.

6. Data Locality

Instead of moving data to where the processing happens, HDFS brings the processing to where the data already is. This reduces network traffic and speeds things up.

7. Snapshots

HDFS supports snapshots—basically, point-in-time backups of the system. If someone accidentally deletes or modifies data, you can roll back to a previous version.

2.2.5 Benefits of Using HDFS

HDFS offers several advantages that make it a preferred choice for big data storage:

1. Cost-Effective

HDFS runs on regular, inexpensive hardware (no fancy servers required). And since it's open-source, you don't pay any licensing fees.

2. Handles Huge Datasets

From structured databases to unstructured files like videos, HDFS can store just about anything—at sizes ranging from megabytes to petabytes.

3. Fast Recovery

If something goes wrong, HDFS automatically detects the issue and recovers using replicated copies. No need to babysit the system.

4. Cross-Platform Support

HDFS works on Linux, Windows, macOS—you name it. It's also portable across different types of hardware.

5. Built for Streaming

If your application needs to constantly read or write large chunks of data (like log monitoring or video streaming), HDFS is optimized for that kind of high-throughput work.

6. Speed

Thanks to its cluster-based design, HDFS can process data at lightning speed—up to 2 GB per second in some setups.

7. Flexible Data Formats

HDFS supports all types of data:

- **Structured** (like SQL tables)
- **Semi-structured** (like XML or JSON)
- **Unstructured** (like images or audio)

It doesn't even require a fixed schema—great for evolving data needs.

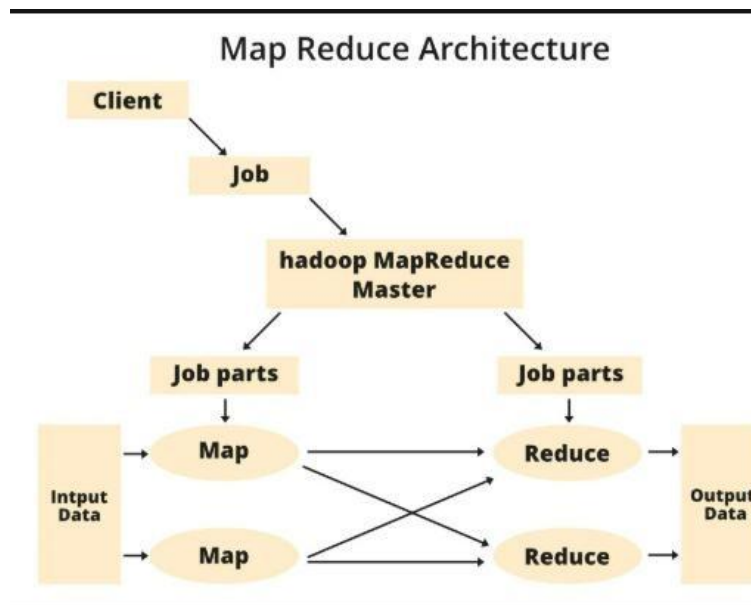
Use Cases

HDFS is widely used in scenarios requiring large-scale data storage and processing, such as:

- Big data analytics and business intelligence
- Centralized data lakes storing mixed data types
- Web server log processing and monitoring
- Machine learning and AI workloads
- Backup systems and disaster recovery

2.3 MapReduce Framework

MapReduce and HDFS are the two major components of Hadoop which makes it so powerful and efficient to use. MapReduce is a programming model used for efficient processing in parallel over large data-sets in a distributed manner. The data is first split and then combined to produce the final result. The libraries for MapReduce is written in so many programming languages with various different-different optimizations. The purpose of MapReduce in Hadoop is to Map each of the jobs and then it will reduce it to equivalent tasks for providing less overhead over the cluster network and to reduce the processing power. The MapReduce task is mainly divided into two phases Map Phase and Reduce Phase.



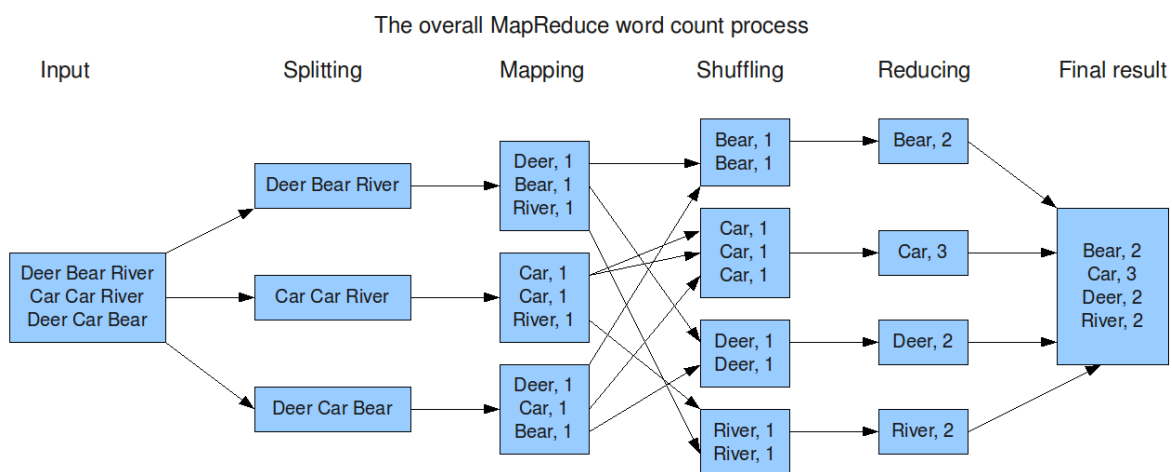
2.3.1 Components of MapReduce Architecture:

- 1. Client:** The MapReduce client is the one who brings the Job to the MapReduce for processing. There can be multiple clients available that continuously send jobs for processing to the Hadoop MapReduce Manager.
- 2. Job:** The MapReduce Job is the actual work that the client wanted to do which is comprised of so many smaller tasks that the client wants to process or execute.
- 3. Hadoop MapReduce Master:** It divides the particular job into subsequent job-parts.
- 4. Job-Parts:** The task or sub-jobs that are obtained after dividing the main job. The result of all the job-parts combined to produce the final output.
- 5. Input Data:** The data set that is fed to the MapReduce for processing.
- 6. Output Data:** The final result is obtained after the processing.

In MapReduce, we have a client. The client will submit the job of a particular size to the Hadoop MapReduce Master. Now, the MapReduce master will divide this job into further equivalent job-

parts. These job-parts are then made available for the Map and Reduce Task. This Map and Reduce task will contain the program as per the requirement of the use-case that the particular company is solving. The developer writes their logic to fulfill the requirement that the industry requires. The input data which we are using is then fed to the Map Task and the Map will generate intermediate key-value pair as its output. The output of Map i.e. these key-value pairs are then fed to the Reducer and the final output is stored on the HDFS. There can be n number of Map and Reduce tasks made available for processing the data as per the requirement. The algorithm for Map and Reduce is made with a very optimized way such that the time complexity or space complexity is minimum.

2.3.2 MapReduce Example:



2.3.3 The MapReduce task is mainly divided into 2 phases i.e. Map phase and Reduce phase.

1. Map

As the name suggests its main use is to map the input data in key-value pairs. The input to the map may be a key-value pair where the key can be the id of some kind of address and value is the actual value that it keeps. The Map() function will be executed in its memory repository on each of these input key-value pairs and generates the intermediate key-value pair which works as input for the Reducer or Reduce() function.

2. Reduce

The intermediate key-value pairs that work as input for Reducer are shuffled and sort and send to the Reduce() function. Reducer aggregate or group the data based on its key-value pair as per the reducer algorithm written by the developer.

2.3.4 Two essential daemons of MapReduce:

1. Job Tracker (Master)

The work of Job tracker is to manage all the resources and all the jobs across the cluster and also to schedule each map on the Task Tracker running on the same data node since there can be hundreds of data nodes available in the cluster.

2. Task Tracker (Worker)

The Task Tracker can be considered as the actual slaves that are working on the instruction given by the Job Tracker. This Task Tracker is deployed on each of the nodes available in the cluster that executes the Map and Reduce task as instructed by Job Tracker.

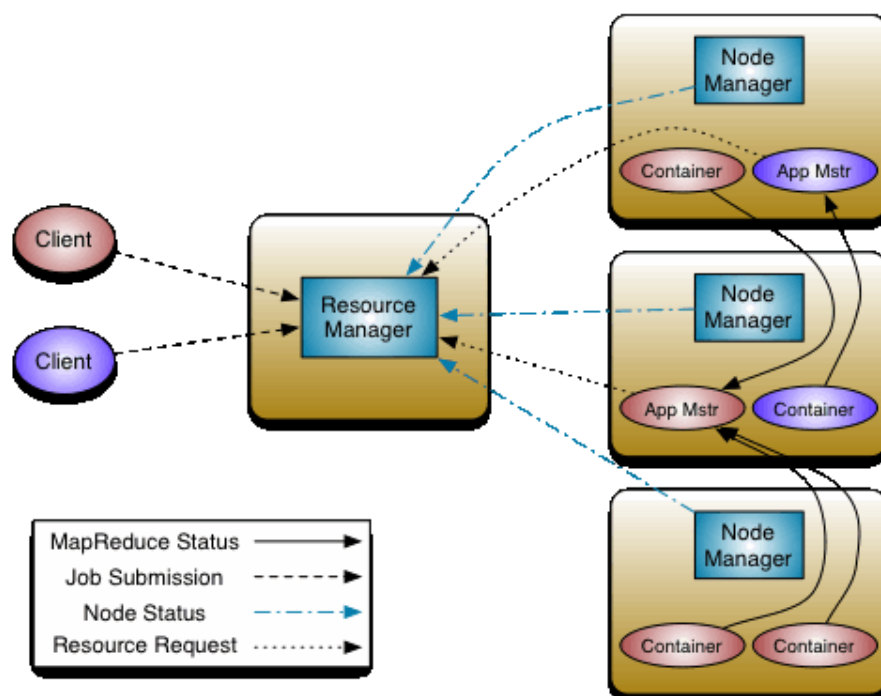
There is also one important component of MapReduce Architecture known as Job History Server. The Job History Server is a daemon process that saves and stores historical information about the task or application, like the logs which are generated during or after the job execution are stored on Job History Server.

2.4 Hadoop YARN

YARN is a core part of the Hadoop ecosystem. Its main job? To manage resources and schedule tasks across the cluster. What makes YARN different is how it separates resource management from job scheduling—splitting these into different components to make the system more flexible and scalable.

How YARN Works – Explained with the Diagram

The diagram outlines how YARN manages resources and runs jobs in a distributed Hadoop cluster. Here's what's happening step by step:



1. Clients Submit Jobs

On the left, you see **clients** submitting jobs to the **ResourceManager** (RM). This is your entry point into the cluster.

- **Job Submission** happens via the dotted black arrow.
 - The RM receives the job and kicks off the next steps.
-

2. ResourceManager Coordinates the Cluster

The **ResourceManager** is the central authority. It doesn't run the job itself but does the following:

- **Keeps track of all cluster resources**
- **Allocates containers** to run applications
- **Communicates with all NodeManagers**

It communicates with **NodeManagers** via **Node Status** updates (shown with dashed blue arrows).

3. ApplicationMaster Gets Launched

For each submitted job, the RM launches an **ApplicationMaster (App Mstr)** inside a **container** on one of the nodes. This is the purple oval you see inside the NodeManager boxes.

- **The App Master is responsible for that specific job only.**
 - It negotiates further resources from the RM (see the dotted blue arrow labeled "Resource Request").
 - Then it tells NodeManagers where to launch the actual work (tasks).
-

4. Containers Run the Job Tasks

Each **NodeManager** (one per machine) is in charge of launching **containers** where the actual job tasks run (Map/Reduce tasks, or any other app).

- Tasks are launched in containers (shown in pink).
 - These containers are isolated environments with specific CPU, memory, etc.
-

5. Monitoring & Communication

Throughout the process:

- The AppMaster talks to the RM for more resources.
- NodeManagers send heartbeat and status updates to the RM.
- The **MapReduce status** (solid black arrow) goes back to the client so they can monitor progress.

Summary of Components from the Figure

Component	Role
Client	Submits the job to the cluster
ResourceManager	Manages cluster resources and launches the AppMaster
ApplicationMaster	Manages the lifecycle of a single job: requests containers and tracks task execution
NodeManager	Manages containers on individual nodes
Containers	Run the actual job tasks

2.5 Concept of Hadoop Applications: Hive, Spark, Sqoop, Pig

The Hadoop ecosystem is a powerful platform designed to handle **big data**—storing, processing, managing, and analyzing it at scale. Hadoop itself is not just a single tool; it's a **collection of tools (components)**, each responsible for a specific function, making it scalable, flexible, and efficient for various data-related operations.

◆ 1. Data Ingestion Tools: Sqoop & Flume

Sqoop – *Structured Data Import/Export*

- Sqoop is used for **transferring structured data** between **relational databases** (like MySQL, Oracle) and the Hadoop ecosystem.
- Example: If you're collecting data from a MySQL database and want to analyze it in Hadoop, Sqoop can **import that data into HBase or Hive**.
- Similarly, Sqoop can **export processed data back** to a database from Hadoop.

Flume – *Unstructured Data Ingestion*

- Flume is best suited for **streaming or semi/unstructured data** like **logs, social media feeds, or real-time sensor data**.
- It collects data continuously and sends it into Hadoop systems like HDFS or HBase.

◆ 2. Data Storage & Modeling: HBase

HBase – *Column-Oriented NoSQL Storage*

- Based on Google's Bigtable, HBase is a **NoSQL database** used to store **massive volumes of structured or semi-structured data**.
 - It stores data in a **column-wise format**, unlike traditional row-based databases, and doesn't require a predefined schema.
 - HBase is optimized for **real-time read/write access** to big data.
-

◆ 3. Data Processing Tools: Spark & Pig

Apache Spark – *Fast In-Memory Processing*

- Spark is a modern alternative to MapReduce for processing data and is **100x faster** due to **in-memory computation**.
- It supports **batch processing, streaming data, machine learning, and graph processing**.
- Spark can be used via multiple languages like **Python, Java, Scala**, and integrates well with Hive, HBase, and others.

Pig – *Simplified Scripting for Data Processing*

- Pig provides a scripting language called **Pig Latin** that simplifies complex data processing tasks.
 - Instead of writing hundreds of lines of Java code for MapReduce, you can write **10–20 lines in Pig**.
 - It's used for **data transformation, filtering, joining, and aggregating** large datasets easily.
 - Ideal for users who are not comfortable with Java-based Hadoop programming.
-

◆ 4. Data Analysis & Query: Hive

Hive – *SQL-like Querying on Hadoop*

- Hive allows you to use **SQL-like queries** to interact with data in Hadoop.
- It converts SQL queries into MapReduce or Spark jobs in the backend.
- Ideal for users familiar with SQL who want to **run ad-hoc queries** or generate **reports** from large datasets.
- Best suited for **structured data**.

◆ 5. Workflow & Cluster Management Tools

Oozie

- A **workflow scheduler** for Hadoop jobs.
- You can define job sequences and automate processes like: "Run job A, then job B at 5 PM daily".

Zookeeper

- A coordination tool used to **manage distributed systems**.
 - Helps in **synchronization, configuration, and group services** across various components.
 - More useful when you're managing **large-scale Hadoop clusters**.
-

◆ 6. Additional Tools

Mahout

- A machine learning library in the Hadoop ecosystem.
- It includes algorithms for **clustering, classification, and recommendation**.

Impala

- A **SQL query engine** that runs **very fast interactive queries** on big data.
- Often used as a **faster alternative to Hive**.

Cloudera Manager / Ambari / Hue

- These are **monitoring and administration tools** to **visualize, manage, and interact** with your Hadoop clusters.
 - For example, **Hue** provides a web-based interface to run queries and manage datasets easily.
-

✅ Summary: Key Functions of Hadoop Applications

Function	Tools Involved
Data Ingestion	Sqoop (structured), Flume (unstructured)
Storage	HBase (NoSQL, columnar storage)
Processing	Spark (fast, in-memory), Pig (scripting)

Querying & Analysis	Hive (SQL-like), Impala (fast queries)
Machine Learning	Mahout
Workflow Scheduling	Oozie
Cluster Coordination	Zookeeper
Monitoring & UI	Cloudera Manager, Ambari, Hue

Focus Tools Summary

- **Hive:** SQL-like querying for structured data.
- **Spark:** Extremely fast data processing, better than traditional MapReduce.
- **Sqoop:** Efficient tool for moving structured data in and out of Hadoop.
- **Pig:** Simplifies writing data transformations with easy scripting.

Q. Write and execute a simple MapReduce program.

 **Goal: Count the number of times each word appears in a text file**

Step 1: Create the Mapper

Create a file called `mapper.py`:

```
#!/usr/bin/env python3
import sys

# Input comes from standard input (stdin)
for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print(f"{word}\t1")
```

- ◆ This splits each line into words and outputs each word with a count of 1.

Step 2: Create the Reducer

Create a file called `reducer.py`:

```
#!/usr/bin/env python3
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split('\t')
    count = int(count)

    if word == current_word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

# Don't forget to output the last word
if current_word == word:
    print(f"{current_word}\t{current_count}")
```

- ◆ This aggregates counts of each word.

Step 3: Prepare Sample Input

Create a text file `input.txt`:

```
hello world
hello mapreduce
hello big data
```

Step 4: Run MapReduce using Hadoop Streaming (Linux/Unix)

```
chmod +x mapper.py reducer.py

hadoop fs -mkdir /input
hadoop fs -put input.txt /input

hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
    -input /input \
```

```
-output /output \  
-mapper mapper.py \  
-reducer reducer.py \  
-file mapper.py \  
-file reducer.py
```

(Path to hadoop-streaming.jar may vary based on installation)

Step 5: View Output

```
hadoop fs -cat /output/part-00000
```

Expected output:

```
big      1  
data     1  
hello    3  
mapreduce 1  
world    1
```

Q1. Install and configure Hadoop on a local machine.

Ans:

To install and configure Hadoop on a local machine (usually in **pseudo-distributed mode**), follow these steps:

On Windows:

Step 1: Install Java (JDK)

1. Download **Java SE Development Kit** (JDK 8 or 11) from:
<https://www.oracle.com/java/technologies/javase-downloads.html>
2. Install it (e.g., C:\Java\jdk1.8.0_XXX).
3. Set **Environment Variables**:
 - JAVA_HOME = C:\Java\jdk1.8.0_XXX
 - Add %JAVA_HOME%\bin to **Path**.

Verify installation:

```
java -version
```

✅ Step 2: Download Hadoop

1. Download Hadoop binary from:
<https://hadoop.apache.org/releases.html>
 2. Extract it (e.g., to C:\hadoop).
-

✅ Step 3: Configure Hadoop Environment Variables

Add these to **System Environment Variables**:

- HADOOP_HOME = C:\hadoop
 - Add %HADOOP_HOME%\bin and %HADOOP_HOME%\sbin to **Path**.
-

✅ Step 4: Configure Hadoop Files (Can be skipped)

Navigate to:

C:\hadoop\etc\hadoop

Update the following files:

core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:/C:/hadoop_data/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:/C:/hadoop_data/hdfs/datanode</value>
  </property>
</configuration>
```

Create directories:

```
mkdir C:\hadoop_data\hdfs\namenode
mkdir C:\hadoop_data\hdfs\datanode
```

mapred-site.xml

Create from template:

```
copy mapred-site.xml.template mapred-site.xml
```

Then add:

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

✅ **Step 5: Add Hadoop Native Binaries (Windows DLLs)**

Hadoop needs **WinUtils.exe** for Windows compatibility.

1. Download winutils.exe (matching your Hadoop version) from:
<https://github.com/cdarlint/winutils>
2. Place it in:
C:\hadoop\bin\winutils.exe

✅ **Step 6: Format NameNode**

Open **Command Prompt** as Administrator:

```
hdfs namenode -format
```

✅ **Step 7: Start Hadoop Services**

Run:

```
start-dfs.cmd  
start-yarn.cmd
```

✅ Step 8: Verify Installation

Check Hadoop daemons using:

```
jps
```

You should see:

```
NameNode  
DataNode  
ResourceManager  
NodeManager
```

✅ Step 9: Access Hadoop Web Interfaces

- **HDFS NameNode UI:**
<http://localhost:9870>
 - **YARN ResourceManager UI:**
<http://localhost:8088>
-

On Linux:

✅ Step 1: Update System and Install Java

Hadoop requires Java. Update your system and install OpenJDK:

```
sudo apt update  
sudo apt install openjdk-11-jdk -y
```

Verify Java installation:

```
java -version
```

✅ Step 2: Create Hadoop User (Optional but Recommended)

It's best practice not to run Hadoop as root:

```
sudo adduser hadoop  
sudo usermod -aG sudo hadoop  
su - hadoop
```

✅ Step 3: Download and Extract Hadoop

Download the latest stable Hadoop from Apache:

```
cd /tmp
wget
https://downloads.apache.org/hadoop/common/stable/hadoop-
X.X.X.tar.gz # Replace X.X.X with the latest version
```

Extract to /usr/local/hadoop:

```
sudo tar -xvzf hadoop-*.tar.gz -C /usr/local/
sudo mv /usr/local/hadoop-* /usr/local/hadoop
```

Give ownership to hadoop user:

```
sudo chown -R hadoop:hadoop /usr/local/hadoop
```

✅ Step 4: Configure Environment Variables

Open ~/.bashrc:

```
nano ~/.bashrc
```

Add these lines at the end:

```
# Hadoop Environment
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Apply changes:

```
source ~/.bashrc
```

✅ Step 5: Configure Hadoop XML Files

Navigate to:

```
cd $HADOOP_HOME/etc/hadoop/
```

Edit the following:

core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
```

```
    </property>
</configuration>
```

hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:/usr/local/hadoop/tmp/dfs/name</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:/usr/local/hadoop/tmp/dfs/data</value>
  </property>
</configuration>
```

Create directories:

```
mkdir -p /usr/local/hadoop/tmp/dfs/name
mkdir -p /usr/local/hadoop/tmp/dfs/data
```

mapred-site.xml (create from template)

```
cp mapred-site.xml.template mapred-site.xml
Add:
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

Step 6: Format HDFS

Run:

```
hdfs namenode -format
```

✔ Step 7: Start Hadoop Services

Start HDFS:

```
start-dfs.sh
```

Start YARN:

```
start-yarn.sh
```

✔ Step 8: Verify Installation

Check running services:

```
jps
```

You should see:

```
NameNode  
DataNode  
ResourceManager  
NodeManager
```

✔ Step 9: Access Hadoop Web UIs

- **HDFS NameNode UI:**
<http://localhost:9870>
 - **YARN ResourceManager UI:**
<http://localhost:8088>
-

Q2. Perform basic HDFS operations (upload, download, delete files).

Ans:

Once Hadoop is configured, we can use the `hdfs dfs` command to interact with HDFS. Here are some basic operations:

1. **Upload a file** to HDFS:

```
hdfs dfs -put localfile.txt /user/hadoop/
```

2. **Download a file** from HDFS to local system:

```
hdfs dfs -get /user/hadoop/localfile.txt .
```

3. **List files and directories** in HDFS:

```
hdfs dfs -ls /user/hadoop/
```

4. **Delete a file or folder** from HDFS:

```
hdfs dfs -rm /user/hadoop/localfile.txt  
hdfs dfs -rm -r /user/hadoop/foldername
```

5. **Create a directory** in HDFS:

```
hdfs dfs -mkdir /user/hadoop/input
```

These operations are used to manage data in the Hadoop Distributed File System.

Chapter 3: Data Visualization with Power BI

3.1 Power BI Overview and Components

Power BI is a powerful business analytics tool developed by Microsoft. It helps you visualize your data, share insights, and make data-driven decisions in an intuitive and user-friendly way. Power BI is a collection of software services, apps, and connectors that work together to turn your unrelated sources of data into coherent, visually immersive, and interactive insights. Your data might be an Excel spreadsheet, or a collection of cloud-based and on-premises hybrid data warehouses. Power BI lets you easily connect to your data sources, visualize and discover what's important, and share that with anyone or everyone you want.

3.1.1 Features of Power BI:

- **Data Visualization:** Create interactive reports and dashboards with a variety of charts, graphs, and visuals.
- **Data Integration:** Connect to multiple data sources such as Excel, databases, cloud services, or APIs, and consolidate the data in one place.
- **Data Transformation:** Clean, shape, and prepare your data for analysis using built-in tools like Power Query.
- **Real-Time Analytics:** Monitor real-time data and set up alerts for actionable insights.
- **Collaboration:** Share reports and dashboards across teams using the Power BI service.
- **AI and Machine Learning:** Use AI capabilities to detect patterns, trends, and outliers in your data.

3.1.2 Why Power BI?

Following are the reasons why Power BI is so popular and needed in the BI domain:

1. Access to Volumes of Data from Multiple Sources

Power BI can access vast volumes of data from multiple sources. It allows you to view, analyze, and visualize vast quantities of data that cannot be opened in Excel. Some of the important data sources available for Power BI are Excel, CSV, XML, JSON, pdf, etc. Power BI uses powerful compression algorithms to import and cache the data within the .PBIX file.

2. Interactive UI/UX Features

Power BI makes things visually appealing. It has an easy drag and drops functionality, with features that allow you to copy all formatting across similar visualizations.

3. Exceptional Excel Integration

Power BI helps to gather, analyze, publish, and share Excel business data. Anyone familiar with Office 365 can easily connect Excel queries, data models, and reports to Power BI Dashboards.

4. Accelerate Big Data Preparation with Azure

Using Power BI with Azure allows you to analyze and share massive volumes of data. An azure data lake can reduce the time it takes to get insights and increase collaboration between business analysts, data engineers, and data scientists.

5. Turn Insights into Action

Power BI allows you to gain insights from data and turn those insights into actions to make data-driven business decisions.

6. Real-time Stream Analytics

Power BI will enable you to perform real-time stream analytics. It helps you fetch data from multiple sensors and social media sources to get access to real-time analytics, so you are always ready to make business decisions.

3.1.3 Advantages Of Power BI

1. **User-friendly interface:** Power BI has an intuitive interface allowing users to visualize and analyze data easily.
2. **Data integration:** Power BI allows users to easily integrate data from various sources, including Excel, SQL Server, and cloud-based sources like Azure and Salesforce.
3. **Customizable dashboards:** Users can create customized dashboards and reports to display data in a way that is meaningful to them.
4. **Real-time data:** Power BI supports real-time data processing, which means users can view up-to-date data in their dashboards and reports.
5. **Collaboration:** Power BI allows users to share their dashboards and reports with others, making collaborating on data analysis projects easy.

3.1.4 Disadvantages Of Power BI

1. **Limited data processing capabilities:** Power BI is not designed for heavy-duty data processing and may struggle with large datasets or complex queries.
2. **Limited customization options:** While Power BI offers a range of customization options, users may find that they are limited in their ability to create truly unique visualizations and reports.
3. **Cost:** Power BI is not a free tool, and users may need to pay for additional features or storage space.

3.2 Connecting to Data Sources

Connecting to data sources in Power BI is the critical first step in any data analysis project, akin to a chef gathering fresh ingredients from various markets to prepare a gourmet meal. Power BI allows users to access a wide range of data sources, from local files to cloud-based platforms, enabling seamless integration of diverse datasets for analysis and reporting.

- **How to Connect (Steps):**
 - Launch **Power BI Desktop** on your computer to begin the process.
 - Navigate to the **Home ribbon** and select **Get Data** to open a menu displaying available data sources.
 - Choose your data source, which could include:
 - **Files:** Excel, CSV, or JSON files stored locally or on a network.
 - **Databases:** SQL Server, MySQL, Oracle, or PostgreSQL.
 - **Cloud Platforms:** Azure SQL Database, Amazon Redshift, or Google BigQuery.
 - **Online Services:** Salesforce, SharePoint, Dynamics 365, or web APIs.
 - Configure the connection based on the source type:
 - For files, browse and select the file location.
 - For databases, provide details like server name, database name, and authentication credentials (username/password).
 - For online services, authenticate using an API key, OAuth, or account credentials.
 - Preview the data in Power BI's interface, selecting specific tables or fields to import.
 - Decide whether to **Load** the data directly into Power BI for immediate use or **Transform Data** to open the Power Query editor for further cleaning and shaping.
 - Once loaded, the data is ready for building reports and visualizations.

3.3 Data Transformation and Modeling

1. Data Transformation:

Data transformation is the process of refining raw data into a usable format, much like a chef preparing ingredients in a kitchen by washing, chopping, and seasoning them before cooking. In

Power BI, transformation is performed in the Power Query editor, ensuring data is clean, consistent, and ready for analysis or visualization.

- **Key Transformation Tasks:**

- **Cleaning:** Correct errors like typos, remove duplicate rows, or fill in missing values (e.g., replacing blank cells with a default value like "Unknown").
- **Restructuring:** Reorganize data, such as pivoting columns to rows or converting data formats (e.g., from XML to a tabular structure).
- **Enrichment:** Enhance datasets by adding new information, like appending country names to a dataset with city codes.
- **Aggregation:** Summarize data for analysis, such as calculating total sales by month from daily records.
- **Normalization/Standardization:** Ensure consistent formats, like converting all dates to "YYYY-MM-DD" or standardizing text case (e.g., all uppercase).
- **Filtering:** Remove irrelevant data, such as excluding outdated records or specific columns not needed for analysis.
- **Integration:** Merge data from multiple sources, resolving conflicts like mismatched column names or duplicate entries.

3. Data Modeling:

Data modeling involves creating a structured blueprint of how data is organized, stored, and related, similar to an architect designing a house to ensure it's functional and efficient. In Power BI, data modeling defines the relationships and structure of datasets to support effective querying and reporting.

- **Key Components:**

- **Entities:** Represent core objects, such as "Customer," "Product," or "Order," that form the foundation of the model.
- **Attributes:** Describe properties of entities, like a Customer's Name, Email, or Phone Number.
- **Relationships:** Define connections, such as a Customer placing multiple Orders, often represented with keys (e.g., CustomerID linking tables).

- **Types of Data Models:**

- **Conceptual Model:** A high-level overview focusing on entities and relationships, like a sketch of a house's layout without technical details.

- **Logical Model:** A detailed design specifying attributes, keys, and relationships, independent of any specific database software.
- **Physical Model:** The actual implementation in a database system (e.g., SQL Server), including tables, columns, data types, and constraints like indexes.

Steps of Data Transformation and Modeling

1. Data Transformation in Power BI (Power Query Editor)

Step 1: Load the Data

- Open Power BI Desktop.
- Click **Home** → **Get Data**.
- Choose your data source (Excel, SQL, CSV, Web, etc.).
- Click **Load** to bring the data into Power BI.

Step 2: Open Power Query Editor

- Click **Transform Data** to launch Power Query Editor.
- This is where you clean and shape your data before analysis.

Step 3: Clean the Data

Use these options to make your data usable:

- **Remove Columns:** Get rid of unnecessary fields.
- **Remove Rows:** Eliminate nulls, errors, or irrelevant data.
- **Replace Values:** Fix typos or standardize naming (e.g., “N/A” → “Null”).
- **Filter Data:** Include only what you need (e.g., sales after 2022).
- **Fill Down / Up:** Fill missing values in a column using nearby data.

Step 4: Transform Data

- **Change Data Types:** Set proper formats (e.g., Date, Whole Number).
- **Split Columns:** Divide values using delimiters (e.g., split full name into first and last).
- **Merge Columns:** Combine fields like "City" and "State" into one.
- **Group By:** Aggregate data (e.g., sum sales by region).
- **Pivot / Unpivot:** Convert between wide and long formats as needed.

Step 5: Create Custom Columns

- Use the **Add Column** tab → **Custom Column**.

- Write formulas using **Power Query M Language** (e.g., if [Sales] > 500 then "High" else "Low").

Step 6: Apply & Close

- After all transformations, click **Close & Apply** to push the cleaned data back to Power BI for modeling and reporting.
-

2. Data Modeling in Power BI

Step 1: Define Relationships

- Go to the **Model view** (Diagram icon).
- Drag and drop fields to link tables (usually by key columns like CustomerID or ProductID).
- Relationships can be:
 - **One-to-Many (1:*)**
 - **Many-to-One (*:1)**
 - **Many-to-Many (:)**
 - Choose **Single or Both** direction depending on the use case.

Step 2: Set Table Roles

- Rename tables for clarity.
- Hide raw or supporting tables that shouldn't be used directly in visuals.
- Tag fact and dimension tables (for your own understanding).

Step 3: Create Calculated Columns

- Use DAX to create new fields within a table.

FullName = Customers[FirstName] & " " & Customers[LastName]

Step 4: Create Measures

- Measures are dynamic calculations like totals, averages, or ratios.

Total Sales = SUM(Sales[Amount])

Profit Margin = DIVIDE(SUM(Sales[Profit]), SUM(Sales[Revenue]))

Step 5: Set Data Formatting

- Format numbers (currency, decimal places).
- Set sorting (e.g., MonthName sorted by MonthNumber).
- Mark date tables as "Date Table" for time intelligence.

Step 6: Validate the Model

- Test relationships with simple visuals (e.g., count of customers by region).
- Check for duplicates, blank joins, or circular dependencies.

✔ Summary of Steps

Phase	Step
Transformation	Load Data → Clean → Filter → Format → Create Columns → Apply
Modeling	Define Relationships → DAX Columns → Measures → Format → Validate

3.4 Creating Visualizations

Creating visualizations in Power BI is the process of transforming data into charts, graphs, or maps to uncover insights, much like an artist painting a picture to tell a story. Visualizations make complex data accessible and understandable for decision-making.

- **Steps to Create Visualizations:**
 - **Connect:** Import data using the “Get Data” option from sources like Excel, databases, or web services.
 - **Prepare:** Use Power Query to clean or transform data, such as removing duplicates or reformatting columns.
 - **Select a Visual:** In the **Report view**, choose a visualization type from the Visualizations pane, such as a bar chart, line graph, pie chart, or map.
 - **Add Fields:** Drag data fields to the visual’s properties (e.g., Axis for categories like “Months,” Values for metrics like “Sales,” or Legend for breakdowns like “Region”).
 - **Customize:** Use the **Format pane** to adjust colors, labels, titles, or fonts to enhance clarity and aesthetics.
 - **Add Interactivity:** Incorporate slicers or filters to allow users to interact with the visual, such as filtering by year or category.

3.5 Sharing and Collaborating with Power BI

Sharing and collaborating in Power BI allows users to distribute insights and work together on reports, similar to a team sharing a collaborative notebook to refine ideas. Power BI provides multiple methods to ensure insights reach the right audience securely and efficiently.

Steps of Sharing and Collaborating with Power BI

1. Publish Report to Power BI Service

After building your report in Power BI Desktop:

- Click **File** → **Publish** → **To Power BI**.
 - Sign in with your Power BI (work or school) account.
 - Choose your workspace (default is *My Workspace*, but better to publish to a shared workspace).
 - Your report and dataset are now in the cloud at app.powerbi.com.
-

2. Create or Use a Workspace

A **workspace** is where teams collaborate on datasets, reports, dashboards, and dataflows.

- Go to **Power BI Service** → **Workspaces**.
 - Click **New Workspace**.
 - Name it and assign permissions:
 - **Admin** – full control.
 - **Member** – edit and publish.
 - **Contributor** – edit only their content.
 - **Viewer** – can only view.
-

3. Share Reports & Dashboards

Option 1: Share with Individuals

- Open your report → Click **Share** (top-right).
- Enter email addresses of team members.
- You can choose:
 - Allow recipients to share.
 - Send email notification.
 - Give access to underlying dataset.

Option 2: Publish to Web (Not Recommended for Sensitive Data)

- From the report menu, select **File** → **Embed report** → **Publish to web**.
 - Public link is generated, but **anyone can view** — no login required.
-

4. Use Apps for Wider Sharing

- Bundle your datasets, reports, and dashboards into a **Power BI App**.
 - Go to the workspace → Click **Create App**.
 - Customize layout, branding, permissions.
 - Share the app link with your audience — they only see the published version.
-

5. Collaborate in Microsoft Teams

- Add Power BI tab to a Teams channel.
 - Share links to reports in chats.
 - Embed Power BI dashboards directly into Teams meetings for real-time review.
-

6. Schedule Data Refresh

- Go to **Datasets** → **Settings** → **Scheduled Refresh**.
 - Set refresh frequency (daily, hourly, etc.).
 - Make sure credentials are updated and the gateway is configured (for on-prem data).
-

7. Version Control with Deployment Pipelines (Premium Feature)

- Create **Development** → **Test** → **Production** stages.
 - Promote content through stages for quality control.
 - Compare changes, manage updates safely in teams.
-

8. Manage Permissions and Access

- Use **Manage Permissions** to:
 - Revoke access.
 - Audit sharing.
 - Control external sharing (can be restricted by admin).
 - Use **Row-Level Security (RLS)** to limit data based on user roles.
-

9. Comment and Tag People

- Open a report → Click **Comment**.
 - Mention users with @ to tag them.
 - Collaborate directly within the report canvas.
-

Summary of Steps

Task	Action
Publish	From Power BI Desktop to Power BI Service
Workspace	Create shared workspaces for team access
Share	Direct share, embed, or publish as app
Collaborate	Use Teams, comments, tagging
Refresh	Automate data updates
Secure	Set permissions, RLS, manage access

Chapter 4: Data Preprocessing for Big Data

4.1 Data Cleaning and Transformation

Data cleaning and transformation are critical first steps before performing any kind of data analysis. Especially in large datasets (like in big data environments), these steps help ensure that the data is accurate, consistent, and usable.

1. Data Cleaning

Data cleaning means identifying and fixing problems in the dataset to improve its quality. Without this step, any analysis or model built on the data could be misleading or completely wrong.

Key Tasks in Data Cleaning:

- **Removing Duplicates**
Get rid of repeated records to prevent inflated or biased results.
 - **Handling Missing Values**
Fill in blanks using methods like:
 - Mean or median value substitution
 - Default placeholders (like "N/A")
 - Deleting rows if necessary
 - **Correcting Inconsistencies**
Standardize values so they're in the same format:
 - Dates (e.g., DD-MM-YYYY vs MM-DD-YYYY)
 - Currencies (e.g., \$, €, ₹)
 - Text casing (e.g., "Nepal" vs "nepal")
 - **Removing Irrelevant Data**
Filter out columns or rows that don't serve any purpose in your analysis.
 - **Fixing Outliers**
Detect and handle data points that are abnormally high or low.
These could be corrected or removed depending on the context.
-

2. Data Transformation

Once the data is clean, it often needs to be transformed into a format that's easier to work with. This step prepares the data for analysis, visualization, or input into machine learning models.

Common Data Transformation Techniques:

- **Normalization and Standardization**
Scale data to a consistent range or distribution (e.g., 0 to 1) to ensure fair comparison across variables.
- **Encoding Categorical Data**
Convert categories like "Male"/"Female" or "Yes"/"No" into numbers so machines can process them.
- **Aggregation and Summarization**
Combine data to generate summaries, such as totals, counts, or averages.
- **Data Type Conversion**
Change data formats — for example:
 - From string to integer
 - From text to date format
- **Pivoting and Reshaping**
Reorganize the data's layout:
 - Turn rows into columns or vice versa
 - Helpful in reports and dashboards

Why Data Cleaning Matters in Big Data Analytics

When dealing with massive datasets, quality becomes more important than quantity. Here's why data cleaning is essential:

1. **Accuracy**
Clean data leads to trustworthy insights.
2. **Performance**
Clean datasets are faster to process and take up less storage.
3. **Better Decisions**
Clear, error-free data helps businesses make the right calls.
4. **Consistency Across Sources**
Big data usually comes from multiple sources (e.g., websites, sensors, transactions). Cleaning aligns them to a single standard.
5. **Improved Machine Learning**
Clean input equals better-trained models and more accurate predictions.

6. Risk Reduction

Dirty data can lead to costly mistakes. Cleaning minimizes that risk.

Popular Tools for Cleaning and Transforming Data

Programming Libraries:

- **Python:** pandas, numpy, scikit-learn
- **R:** dplyr, tidyr

Big Data Tools:

- **Apache Spark**
- **Hadoop**

ETL (Extract, Transform, Load) Tools:

- **Talend**
- **Informatica**

Spreadsheets:

- **Microsoft Excel**
- **Google Sheets**

4.2 Handling Missing and Inconsistent Data

Missing and inconsistent data are two of the biggest troublemakers in any dataset. If not handled properly, they can ruin your analysis, skew your results, and lead to bad decisions. Let's break down what they are and how to deal with them.

1. Understanding Missing Data

Missing data means some values just aren't there — maybe a survey question was skipped, or a sensor failed to record.

Types of Missing Data:

- **MCAR (Missing Completely at Random)**
The missingness is totally random and unrelated to any value in the dataset.
→ *Example: A data entry clerk accidentally skips a few rows.*

- **MAR (Missing at Random)**

The missingness is related to *observed* data, but not the missing part itself.

→ *Example: Income data is missing more often for younger people.*

- **MNAR (Missing Not at Random)**

The missingness depends on the missing value itself.

→ *Example: People with very high income choose not to disclose it.*

🔗 *Why this matters:*

Knowing the type helps you pick the right way to handle the missing data — some techniques work only for MCAR or MAR.

2. How to Handle Missing Data

Here are the go-to strategies:

a. Identify the Type of Missingness

First step: figure out whether it's MCAR, MAR, or MNAR. This shapes everything that follows.

b. Remove Missing Data (Only If Safe!)

If only a few rows or columns have missing values, and they're not important, you can drop them. But don't do this blindly — you might lose valuable info.

c. Impute (Fill) Missing Values

- **Mean/Median/Mode Imputation** – Fill missing values with average or most common value.
- **Predictive Imputation** – Use models like regression or k-NN to estimate missing values.
- **Interpolation/Extrapolation** – Estimate based on trends (useful in time-series data).

d. Use Algorithms That Handle Missing Data

Some models like decision trees, Random Forest, and XGBoost can handle missing values natively.

e. Flag Missing Values

Add an indicator column (like `is_missing = 1`) to track where data was missing — useful for analysis.

f. Use Domain Knowledge

Sometimes, only an expert knows what should happen with a missing value. When in doubt, ask.

3. Understanding Inconsistent Data

Inconsistent data means values that contradict each other or don't follow a standard format.

What Causes It:

- **Mismatched Formats** – e.g., “01-02-2025” vs “2025/02/01”
- **Contradictions** – e.g., One row says “Male”, another says “Female” for the same person.
- **Data Entry Errors** – Typos, misspellings, or wrong values.

4. How to Handle Inconsistent Data

Fixing inconsistency is about enforcing rules and standards:

a. Data Profiling

Run diagnostics to spot anomalies, odd patterns, or unusual distributions.

b. Standardization

Convert everything to a common format — same date formats, same units (e.g., meters vs feet).

c. Apply Validation Rules

Enforce rules like:

- Age must be between 0 and 120
- Phone numbers must be 10 digits
- Dates must follow a standard format

d. Use Automated Cleaning Tools

Tools like OpenRefine or scripts in pandas can catch and correct inconsistent formats.

e. Get Help from Experts

For tricky contradictions, use domain experts to decide what's right.

5. Why It Matters: Impact of Missing & Inconsistent Data

Missing Data Can Cause:

- ▼ **Lower Accuracy** – Incomplete data weakens analysis.
- ⚖️ **Bias** – Systematic missingness skews results.
- 🔍 **Loss of Detail** – Less granularity in insights.
- 🧩 **More Complexity** – Requires extra steps and tools to handle.

Inconsistent Data Can Cause:

-  **Analysis Errors** – Contradictions lead to confusion.
-  **Poor Data Quality** – Garbage in, garbage out.
-  **Integration Issues** – Merging multiple sources becomes a nightmare.
-  **Slower Decisions** – Fixing takes time and effort.

6. Tools to Handle Missing and Inconsistent Data

Here's what the pros use:

Python Libraries

- pandas, NumPy, scikit-learn

R Packages

- dplyr, tidyr, mice

Specialized Tools

- **Data Cleaning:** OpenRefine, Trifacta
- **ETL Platforms:** Talend, Informatica
- **Spreadsheets:** Microsoft Excel, Google Sheets

These tools let you clean, reshape, validate, and analyze data with minimal manual effort.

4.3 Introduction to ETL (Extract, Transform, Load) Processes

ETL stands for **Extract, Transform, Load** — a foundational process in data engineering and analytics. It's how raw, scattered data gets turned into clean, usable information stored in a central system (like a data warehouse or data lake).

1. What is ETL?

ETL is the process of:

- **Extracting** data from various sources,
- **Transforming** it into a clean and usable format,
- **Loading** it into a system for analysis (like a database, data warehouse, or cloud storage).

It's the backbone of most data pipelines and business intelligence workflows.

2. Step-by-Step Breakdown of ETL

✓ A. Extract: Getting the Data

Goal:

Collect raw data from multiple sources, including databases, APIs, files, and logs.

Common Data Sources:

- Relational databases (MySQL, Oracle)
- APIs and Web Services (REST, GraphQL)
- Flat files (CSV, JSON, XML)
- Spreadsheets, logs, cloud services

Key Points:

- Maintain data accuracy during extraction
- Support different formats and protocols (ODBC, JDBC)
- Handle both real-time and batch data streams

Popular Extraction Tools:

- SQL queries
 - Python libraries: pandas, requests
 - Apache NiFi
 - Talend
-

✓ B. Transform: Cleaning and Structuring the Data

Goal:

Convert raw data into a clean, consistent format ready for use.

Common Transformation Tasks:

- **Cleaning** – Remove duplicates, fix errors, handle missing data
- **Aggregation** – Summarize (e.g., total sales by region)
- **Enrichment** – Merge datasets, create new columns
- **Normalization** – Standardize formats and scales
- **Filtering** – Keep only what's needed; remove outliers

Key Considerations:

- Improve overall data quality

- Handle schema changes and enforce correct data types
- Must scale to handle large datasets efficiently

Popular Transformation Tools:

- Python: pandas, PySpark
 - SQL
 - Apache Spark
 - Informatica
 - dbt (Data Build Tool)
-

✓ C. Load: Delivering the Final Data

Goal:

Insert transformed data into the destination system.

Target Systems:

- Data warehouses (e.g., Snowflake, Redshift)
- Databases (PostgreSQL, MySQL)
- Data lakes (Amazon S3, Hadoop HDFS)

Loading Methods:

- **Full Load** – Dump all data every time (used for small/initial datasets)
- **Incremental Load** – Only send new or changed data (efficient and preferred for large systems)

Key Considerations:

- Ensure fast, accurate data loading
- Validate integrity after loading
- Handle schema matching and logging of errors

Popular Loading Tools:

- SQL bulk loaders
 - Apache Airflow
 - AWS Glue
 - Google BigQuery
-

3. ETL Process Summary Table

Step	Purpose	Common Tools
Extract	Collect data from different sources	SQL, Python, Apache NiFi, Talend
Transform	Clean, enrich, and reformat the data	pandas, PySpark, Spark, SQL, dbt
Load	Store processed data into final destination	Airflow, AWS Glue, BigQuery

4. Why ETL Matters: The Role in Data Integration

ETL isn't just a tech task — it's a key part of any data-driven operation. Here's what it enables:

- 1. Unifies Data Sources**
Pulls in data from different systems and puts it in one place.
- 2. Ensures Data Quality**
Cleans and standardizes so everything matches and makes sense.
- 3. Standardizes Formats**
Converts messy formats into something analysis tools can use.
- 4. Prepares Data for Use**
Gets the data ready for dashboards, reporting, machine learning, and decision-making.

4.4 Concept of Data Integration and Aggregation

Data integration and aggregation are two core building blocks in managing and analyzing data across systems. Integration is about bringing everything together from different sources. Aggregation is about summarizing it to make sense of the bigger picture.

4.4.1 Data Integration

What is Data Integration?

Data integration is the process of **combining data from multiple sources** into a unified, consistent format. It feeds data into operational systems (like business apps) and analytical platforms (like data warehouses or BI dashboards).

It's essential for:

- **Eliminating data silos**
- **Making data accessible and usable**
- **Powering analytics and real-time decisions**

Whether the data is structured (from databases) or unstructured (from emails, images, logs), integration ensures that everything speaks the same language.

Why Data Integration Matters

- **✔ Better Decision-Making**
Clean, consistent data = smarter business calls.
- **✔ Simpler Analytics Access**
Data scientists and analysts don't have to hunt for scattered data — it's all in one place.
- **✔ Fewer Data Silos**
Departments can share useful data across the company.
- **✔ Faster Workflows**
Users spend less time cleaning or searching for data.
- **✔ More Data-Driven Operations**
Integrated data leads to more informed planning and execution.

Common Approaches to Data Integration

Method	Description
ETL	Extract → Transform → Load into a data warehouse or database
Data Virtualization	Create a live, unified view of data across systems without moving it
API Integration	Use APIs to connect systems and enable data exchange in real-time

Types of Data Integration

Here's how integration can be done — depending on the use case:

- **a. Manual Integration**
Users gather and combine data manually (e.g., in Excel). Time-consuming, error-prone.
- **b. ETL (Extract, Transform, Load)**
Most common method — data is processed and stored in a warehouse.
- **c. ELT (Extract, Load, Transform)**
Data is first loaded into storage, then transformed. Ideal for cloud platforms.
- **d. Data Virtualization**
No data movement — just a live view across systems.

- **e. Middleware Integration**
Software sits between systems to sync data in real-time.
 - **f. Streaming Data Integration**
Real-time integration for live feeds (e.g., IoT devices, sensors).
 - **g. Application-Based Integration**
Systems like CRM and ERP directly exchange data through APIs.
 - **h. Common Data Storage (Data Warehousing)**
Store all integrated data in one place for analysis and reporting.
-

4.4.2 Data Aggregation

What is Data Aggregation?

Aggregation is about **collecting and summarizing data** from different sources. It helps simplify complex datasets and highlight key insights like totals, averages, or trends.

It's often part of ETL or reporting processes and is used to:

- Simplify large datasets
 - Detect trends and patterns
 - Support faster decision-making
-

Common Aggregation Functions

Function	Description
Sum	Adds up values (e.g., total sales)
Average	Calculates the mean
Count	Counts records or entries
Max/Min	Finds highest or lowest value
Group By	Groups data by category (e.g., region, product)

Example:

Region	Sales
East	100
West	150
East	200

→ After aggregation by Region:

Region	Total Sales
East	300
West	150

4.4.3 Principles of Data Integration

Here are the key principles that make integration effective:

1. **Data Consistency**
Resolve duplicates or conflicts between datasets.
2. **Standardization**
Use common formats, units, and field names.
3. **Interoperability**
Systems should be able to talk to each other and share data smoothly.
4. **Data Quality Assurance**
Fix inaccuracies, missing values, and redundancies as part of integration.
5. **Metadata Management**
Keep clear documentation of where the data comes from and how it's structured.
6. **Scalability**
Your integration process should scale as your data grows.

4.4.4 Principles of Data Aggregation

These principles help guide how to summarize data effectively:

1. **Summarization**
Combine multiple data points into useful summaries (totals, averages, etc.)
2. **Group-Based Aggregation**
Organize data into categories to spot patterns.
3. **Data Compression**
Shrink data size without losing key insights.
4. **Filtering**
Remove unnecessary, outdated, or redundant information.
5. **Timeliness**
Keep the aggregated data up to date — especially important in real-time dashboards.

6. **Context Awareness**

Maintain the meaning of data during aggregation (don't lose context when summarizing).

4.5 Data Reduction and Feature Selection

4.5.1 Data Reduction

Data reduction is the process of simplifying or condensing large datasets while preserving the core information needed for analysis. In big data contexts, this step is essential to make computations manageable, reduce storage costs, and streamline analysis workflows without sacrificing accuracy.

Key Techniques in Data Reduction:

1. **Dimensionality Reduction:**

- *Principal Component Analysis (PCA)*: Transforms high-dimensional data into fewer dimensions by capturing the most variance.
- *t-Distributed Stochastic Neighbor Embedding (t-SNE)*: Useful for visualizing complex, high-dimensional data in 2D or 3D.
- *Linear Discriminant Analysis (LDA)*: Optimized for classification by maximizing separability among known categories.

2. **Data Aggregation:**

Combines data from multiple records or sources into summarized forms like totals or averages, especially across time intervals or categories.

3. **Sampling:**

Selects a statistically representative subset of the full dataset to work with, reducing size while preserving patterns and distributions.

4. **Compression:**

Uses encoding algorithms or data truncation techniques to reduce file size for storage and transmission (e.g., ZIP compression, quantization).

5. **Feature Selection:**

Retains only the most meaningful input variables for analysis, improving model performance and reducing overfitting.

6. **Clustering:**

Groups similar data points into clusters and uses those as summaries or representatives of broader patterns.

4.5.2 Feature Selection

Feature selection involves identifying and retaining the most relevant attributes in a dataset. It simplifies models, reduces overfitting, improves accuracy, and increases interpretability — particularly important in machine learning and predictive modeling tasks.

Common Methods of Feature Selection:

1. Filter Methods:

- Evaluate features independently from any learning algorithm.
- *Correlation Coefficient*: Assesses linear relationships between features and target variables.
- *Chi-Square Test*: Measures dependence between categorical variables.

2. Wrapper Methods:

- Use model performance to assess subsets of features.
- *Recursive Feature Elimination (RFE)*: Iteratively removes the least impactful features.
- *Forward Selection*: Adds features incrementally based on performance gains.
- *Backward Elimination*: Starts with all features and removes the least useful ones.

3. Embedded Methods:

- Integrate feature selection within the model training process.
- *Lasso Regression*: Shrinks coefficients of irrelevant features to zero.
- *Decision Trees / Random Forest*: Assign importance scores to features during training.

4. Dimensionality Reduction (Optional Alternative):

- Although not technically selection, methods like PCA reduce the number of features by combining them into a smaller set of composite variables.

4.5.3 Need for Data Reduction in Big Data Analysis

Data reduction is critical in big data systems, where datasets are massive in scale and complexity. Without reduction, analysis becomes inefficient, expensive, and slow.

Why Data Reduction Is Necessary:

1. Efficiency in Processing:

Smaller datasets enable faster algorithm execution and reduce the time required for data preparation and analysis.

2. **Storage Savings:**

With less redundant or unnecessary data, storage systems are used more effectively, reducing infrastructure costs.

3. **Improved Insight Quality:**

By removing noise and focusing on meaningful variables, analysis results become more accurate and reliable.

4. **Support for Real-Time Applications:**

In time-sensitive domains like fraud detection or stock analysis, data reduction supports faster, more responsive decision-making.

5. **Cost Efficiency:**

Reduced computational requirements translate into lower costs for data transport, processing, and storage.

6. **Scalability:**

As data volumes grow, reduced datasets help maintain performance and scalability of analytics systems.

Chapter 5: Data Analysis Using Python Libraries

5.1 Introduction to Data Analysis and Visualization

In today's data-driven world, the ability to understand, analyze, and visualize data is a crucial skill across disciplines—from business and healthcare to engineering, social sciences, education, and public administration. Every industry generates vast amounts of data, but raw data alone has little value unless it is properly interpreted. This is where **data analysis** and **data visualization** come into play. These two interconnected disciplines transform raw data into meaningful insights, supporting informed decision-making and strategic planning.

5.1.1 Data Analysis

Data Analysis refers to the process of inspecting, cleaning, transforming, and modeling data to uncover useful information, draw conclusions, and support decision-making. It involves applying statistical techniques, logical reasoning, and computational methods to identify trends, patterns, relationships, and anomalies within datasets.

Key Components:

- Data cleaning (handling missing values, inconsistencies)
- Statistical summarization (mean, median, standard deviation)
- Pattern recognition (correlations, clusters, trends)
- Modeling and forecasting (regression, classification)

Types of Data Analysis include:

- **Descriptive Analysis** – Summarizes what has happened
- **Inferential Analysis** – Generalizes findings from a sample to a population
- **Exploratory Analysis** – Identifies patterns without prior assumptions
- **Diagnostic Analysis** – Determines why something happened
- **Predictive Analysis** – Forecasts future outcomes
- **Prescriptive Analysis** – Recommends actions to take

#Types of Data Analysis

(Think: "DIE-DPP" – Descriptive, Inferential, Exploratory, Diagnostic, Predictive, Prescriptive)

1. Descriptive Analysis – “What happened?”

- **Goal:** Summarize and describe data.

- **Tools:** Mean, median, mode, standard deviation.
 - **Use:** To spot basic trends and patterns.
✓ *Think: A summary of facts.*
-

2. Inferential Analysis – “What can we say about the population?”

- **Goal:** Make predictions or generalizations from a sample.
 - **Tools:** Hypothesis testing, confidence intervals, regression.
 - **Use:** To make educated guesses about larger groups.
✓ *Think: Small sample → Big conclusions.*
-

3. Exploratory Data Analysis (EDA) – “What’s interesting in the data?”

- **Goal:** Explore data without assumptions.
 - **Tools:** Visuals, charts, basic stats.
 - **Use:** To uncover patterns or surprises in the data.
✓ *Think: Detective work with data.*
-

4. Diagnostic Analysis – “Why did it happen?”

- **Goal:** Find the root cause of events.
 - **Tools:** Correlation, regression, ANOVA.
 - **Use:** To understand cause-effect relationships.
✓ *Think: Doctor diagnosing a condition.*
-

5. Predictive Analysis – “What will happen next?”

- **Goal:** Forecast future outcomes.
 - **Tools:** Machine learning, time series, predictive models.
 - **Use:** Sales forecasting, risk prediction, behavior modeling.
✓ *Think: Crystal ball using past data.*
-

6. Prescriptive Analysis – “What should we do about it?”

- **Goal:** Suggest the best action to take.

- **Tools:** Optimization, AI, business rules.
 - **Use:** Decision-making, resource planning.
✓ *Think: GPS giving the best route.*
-

#Data Analysis Methods

(Think: “QQ” – Qualitative vs Quantitative)

1. Qualitative Data Analysis – Non-numerical insights

- Deals with **words, images, behaviors, and observations**.
 - **No numbers**, more **interpretation**.
 - **Common methods:**
 - **Content Analysis:** Study of text/speech behavior.
 - **Narrative Analysis:** Analyzing stories/interviews.
 - **Grounded Theory:** Building theories from case studies.
✓ *Think: “Tell me a story” approach.*
-

2. Quantitative Data Analysis – Numbers and statistics

- Deals with **numerical data** and **statistics**.
 - **Objective and measurable**.
 - **Common methods:**
 - **Hypothesis Testing:** Proving or disproving assumptions.
 - **Mean (Average):** Central value of a data set.
 - **Sample Size Analysis:** Using small samples to reflect the whole.
✓ *Think: “Show me the numbers.”*
-

5.1.2 Data Visualization

Data Visualization is the graphical representation of data and analytical results. It uses visual elements such as charts, graphs, plots, maps, and dashboards to present data in an intuitive and accessible format. Visualization helps simplify complex datasets and supports faster and more effective interpretation of data.

Common Visualization Tools:

- Bar Charts, Line Graphs, Pie Charts
- Scatter Plots, Heat Maps, Histograms
- Dashboards (Tableau, Power BI)
- Interactive web-based visualizations (D3.js, Plotly)

5.1.3 Importance of Data Analysis and Visualization

1. Better Decision-Making

Data analysis and visualization equip organizations with evidence-based insights. Instead of relying on intuition or outdated methods, decision-makers can act based on solid facts derived from data. For instance, a business can identify which product is underperforming, or a hospital can detect patterns in patient readmission rates.

2. Pattern and Trend Identification

These processes help uncover hidden patterns, seasonal trends, and correlations that are not obvious in raw data. For example, a retail company might discover that sales of certain products increase during specific times of the year, leading to better inventory planning.

3. Real-Time Monitoring and Feedback

In domains like healthcare, cybersecurity, and finance, real-time data analysis and visualization enable quick responses to critical issues. Dashboards that update in real-time can alert users to anomalies, security breaches, or urgent system failures.

4. Improved Communication of Insights

Data visualization converts complex datasets into clear visuals that can be easily understood by non-technical audiences. This makes it easier to communicate findings to stakeholders, clients, policy-makers, or the general public.

5. Supports Research and Innovation

In academic and scientific research, data analysis is key to hypothesis testing, discovery, and theory development. Visualization tools enhance the exploration of research data and support transparent, reproducible research practices.

5.1.4 Applications Across Various Fields

1. Business and Marketing

- Analyzing customer behavior and preferences
- Monitoring key performance indicators (KPIs)

- Predicting market trends and customer churn

2. Healthcare

- Monitoring disease outbreaks
- Managing patient records and outcomes
- Optimizing hospital resource allocation

3. Education

- Tracking student performance data
- Personalizing learning through analytics
- Evaluating curriculum effectiveness

4. Public Policy and Government

- Understanding population statistics and census data
- Visualizing crime or economic indicators
- Supporting evidence-based policy decisions

5. Environmental Science

- Monitoring climate change indicators
- Visualizing pollution and biodiversity data
- Predicting natural disasters

6. Engineering and Manufacturing

- Predictive maintenance of machines
- Quality control through real-time monitoring
- Process optimization using sensor data

5.2 Concept of python library and bigdata analysis environment

1. What Is a Python Library?

A **Python library** is a collection of reusable modules and functions designed to simplify and accelerate software development. In data science, libraries provide ready-made tools for tasks like data manipulation, numerical computing, visualization, and machine learning. Instead of building algorithms from scratch, analysts and scientists rely on mature, community-tested libraries that are efficient, reliable, and optimized.

2. Why Python for Big-Data Analysis?

Python has become the go-to language for data work due to:

- A **rich ecosystem** of specialized libraries for every step of the data pipeline.
- **Ease of use and readability**, which allow fast prototyping and collaboration.
- **Strong integration** with distributed and cloud systems like Hadoop, Spark, BigQuery, and Redshift.

3. Essential Python Libraries for Big Data

Library	Purpose	Key Features
NumPy	Numerical computing	Multidimensional arrays, vectorized math, linear algebra support
Pandas	Tabular data manipulation	DataFrames, grouping, cleaning, time-series
SciPy	Scientific computing	Optimization, signal processing, integration
Matplotlib	Static and interactive plotting	Highly customizable plots, foundation for many others
Seaborn	Statistical visualization	Stylish, high-level plots built on Matplotlib
Plotly / Bokeh / Altair	Web-based interactive visuals	Dashboards, zoom, hover tools
Dask	Parallel, out-of-core computing	Scales pandas/NumPy workflows, integrates with clusters
Vaex	Fast exploration of massive datasets	Lazy evaluation, histogram & visualization support
PySpark	Distributed big-data processing via Spark	SQL, streaming, MLlib, scalable to clusters
Scikit-learn	Classical machine learning	Regression, classification, clustering

4. Environment for Big-Data Analysis

A **big-data analysis environment** integrates these libraries with systems and platforms that can handle large-scale, distributed, and real-time data workloads:

- **Apache Spark** with PySpark—distributes computation across clusters for fast, scalable processing.
- **Dask clusters** extend local workflows to multi-machine setups.

- **Cloud data warehouses** like Google BigQuery or Amazon Redshift are accessed via Python SDKs or SQL for high-volume analytics.
- **Interactive notebooks** (e.g., Jupyter) provide consoles for exploration, code, results, and visuals in one interactive environment.

Together, these form a powerful stack for exploring, modeling, and visualizing massive datasets.

5. How It All Fits Together

1. **Local Prototyping:** Begin with NumPy, Pandas, Matplotlib/Seaborn in Jupyter to clean, explore, and visualize data interactively.
2. **Scaling Up:** Move to Dask or Vaex for larger-than-memory datasets using familiar APIs.
3. **Full Big-Data Processing:** Employ PySpark on cluster environments for heavy-duty distributed computing.
4. **Machine Learning Integration:** Use Scikit-learn for moderate datasets and pipelines, or Spark MLlib for full-scale models.
5. **Interactive Delivery:** Share findings via Plotly- or Bokeh-powered dashboards, potentially embedded in web apps.

5.3 NumPy and Statistical functions

1. Introduction to NumPy

NumPy (Numerical Python) is a powerful open-source Python library used for numerical and scientific computing. It provides:

- The **ndarray** (N-dimensional array) for efficient data storage and manipulation,
- Mathematical and **statistical functions** for quick analysis,
- Support for **vectorized operations**, making code faster and cleaner than traditional Python loops.

NumPy forms the foundation for many other Python libraries like Pandas, SciPy, Scikit-learn, and TensorFlow.

2. Setting Up NumPy

Before using NumPy, you need to install it. Use the following command:

```
pip install numpy
```

Then, in your Python script or Jupyter notebook, import it:

```
import numpy as np
```

3. Why Use NumPy for Statistics?

NumPy is widely used in data science, machine learning, and scientific computing because:

- It handles large datasets efficiently.
- Functions are written in C under the hood for performance.
- It allows easy integration with visualization and ML libraries.
- Statistical functions are built-in and optimized for large-scale analysis.

4. Common Statistical Functions in NumPy

Here's a breakdown of important NumPy statistical functions with simple explanations:

Function	Purpose	Example
np.mean()	Mean (average)	np.mean(data)
np.median()	Middle value	np.median(data)
np.std()	Standard deviation (spread of data)	np.std(data)
np.var()	Variance (measure of dispersion)	np.var(data)
np.min() / np.max()	Minimum / Maximum value	np.min(data)
np.ptp()	Range (max - min)	np.ptp(data)
np.percentile()	Finds specific percentile	np.percentile(data, 75)
np.nanmean() etc.	Ignore NaN values while computing statistics	np.nanmean(data_with_nan)

5. NaN-Safe Statistical Functions

Real-world data often contains **missing values (NaN)**. NumPy handles them with special versions:

- np.nanmean()
 - np.nanstd()
 - np.nanmin()
 - np.nanpercentile()
- These functions skip NaN values instead of failing.

```
data = np.array([10, 20, np.nan, 30])
np.nanmean(data)  # Outputs 20.0
```

6. Axis Parameter: Row vs Column Stats

NumPy functions can operate across rows or columns:

```
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6]])

np.mean(array_2d, axis=0)  # Column-wise: [2.5, 3.5, 4.5]
np.mean(array_2d, axis=1)  # Row-wise:     [2.0, 5.0]
```

- axis=0: Down the columns (vertical)
 - axis=1: Across the rows (horizontal)
-

7. Example Program

```
import numpy as np

# Dataset
data = np.array([10, 20, 30, 40, 50, np.nan])

print("Mean:", np.nanmean(data))
print("Median:", np.nanmedian(data))
print("Standard Deviation:", np.nanstd(data))
print("Variance:", np.nanvar(data))
print("Minimum:", np.nanmin(data))
print("Maximum:", np.nanmax(data))
print("Range (ptp):", np.ptp(data))
print("75th Percentile:", np.nanpercentile(data, 75))
```

```
import numpy as np

data = np.array([10, 41, 45, 25,
                 16, 98, 78, 1, 6, 58, 41, 98, 52, 63])

print("Mean: ", np.mean(data))
print("Median: ", np.median(data))
print("Minimum: ", np.min(data))
print("Maximum: ", np.max(data))
print("Standard Deviation: ", np.std(data))
```

```
print("Variance: ", np.var(data))
print("Range: ", np.ptp(data))

print("number of elements: ", np.size(data))
```

8. Summary: Key Points to Remember

- **NumPy is the base library** for numerical and statistical analysis in Python.
- It provides fast, memory-efficient array operations.
- **Statistical functions like mean, median, std, and percentile** are easy to use.
- Special NaN-safe functions help deal with incomplete datasets.
- Use the **axis parameter** to apply functions across rows or columns.

5.4 Pandas for Data Handling

1. What is Pandas?

Pandas is an open-source Python library created for powerful and flexible data manipulation and analysis. It extends NumPy by offering two core data structures:

- **Series:** A one-dimensional labeled array containing any data type.
- **DataFrame:** A two-dimensional, spreadsheet-like structure composed of multiple Series with shared indices.

Developed by Wes McKinney in 2008, Pandas fills the gap between Python and R's data-frame capabilities, enabling efficient handling of structured data

2. Setup and Installation

Install using:

```
pip install pandas
```

Import in Python or Jupyter:

```
import pandas as pd
```

Pandas supports a wide range of data formats—CSV, Excel, JSON, SQL, HDF5—making data ingestion seamless.

3. Data Input and Output (I/O)

Pandas supports reading and writing data in numerous formats:

- **CSV:** `pd.read_csv()` and `.to_csv()`
- **Excel:** `pd.read_excel()` and `.to_excel()`
- **SQL:** via SQLAlchemy, e.g., `pd.read_sql()` and `.to_sql()`
- **JSON, HDF5, and web APIs:** `pd.read_json()`, `.to_hdf()`, or converting JSON responses to DataFrames.

This makes Pandas a versatile tool for data ingestion from almost any data source

4. Core Data Structures

Series

- One-dimensional labeled array
 - Comparable to a column in a table
 - Supports arithmetic, alignment by index, and missing values handling
-

DataFrame

- Two-dimensional, table-like structure with rows and columns
 - Similar to SQL tables or Excel sheets
 - Supports indexing, slicing, and Boolean filtering
-

5. Essential Data Handling Functions

a. Loading & Inspecting Data

- `pd.read_csv()`, `read_excel()`, `read_sql()` to load
- `df.head()`, `.tail()`, `.shape`, `.dtypes`, `.describe()`, `.isnull().sum()` for overview

b. Indexing & Filtering

- `df['col']`, `df[['col1', 'col2']]`, `df.iloc[0]`, `df.loc[...]`, and condition-based filters

c. Cleaning & Preprocessing

- Missing data: `df.dropna()`, `df.fillna(value)`, `df.interpolate()`
- Duplicates: `df.drop_duplicates()`

- Type conversion: `df['col'] = df['col'].astype(type)`

d. Data Transformation

- Creating new columns: `df['new'] = df['A'] / df['B']`
- Sorting: `df.sort_values(by='col', ascending=False)`

e. Aggregation & Grouping

- `df.groupby('col').mean()`
- Aggregate multiple functions: `.agg({'col': ['sum', 'mean']})`
- Pivot tables: `pd.pivot_table(df, index=..., columns=..., values=..., aggfunc=...)`

f. Time-Series Support

- Datetime parsing and slicing: `df['2023-01':'2023-02']`
- Frequency conversion, moving averages, rolling windows

g. String and Advanced Data Handling

- Vectorized string methods: `df['col'].str.contains(), split, lower/upper`
 - Improved in Pandas 2.0 with Apache Arrow backend and new StringDtype

6. Key Features of Pandas 2.x

- **Nullable data types** allow precise handling of missing values with less memory footprint
- Integration with **Apache Arrow** for faster string handling, time series, and memory efficiency
- **Copy-on-Write** support improves performance safety during indexing operations
- Support for **custom aggregations**, flexible transforms, improved `groupby()`, and advanced StringDtype

7. When and Why to Use Pandas

- **Exploratory Data Analysis (EDA)**: Quickly inspect and summarize data
- **Data Cleaning & Preparation**: Handle missing data, filter, merge datasets
- **Feature Engineering**: Create new variables, encode, transform
- **Statistical Summaries**: Get descriptive stats, correlations, pivot tables

- **Time Series & Financial Processing:** Handle real-world date/time data
- **Prototyping & Integration:** Integrates smoothly with visualization and ML tools

8. Practical Example: Sales Data

Date	Region	Sales
8/1/2025	East	200
8/2/2025	East	NaN
8/3/2025	West	300
8/4/2025	East	200
8/5/2025	West	300
8/5/2025	West	300

```
import pandas as pd

#1 Read Excel File
df = pd.read_excel("sales_data.xlsx")
print("\n*****Original Data Frame*****\n")
print(df)

#2 Handle Missing Values
df['Sales'] = df['Sales'].interpolate(method='linear')
print("\n*****Handle Missing Values*****\n")
print(df)

#3 Remove Duplicates
df = df.drop_duplicates()
print("\n*****Duplicates Removed*****\n")
print(df)

#4 Add Column
df['Sales_with_VAT'] = df['Sales'] + df['Sales']*0.13
print("\n*****Column Added*****\n")
print(df)

#5 Sort by Sales
df = df.sort_values(by='Sales', ascending=True)
print("\n*****Sorted*****\n")
print(df)

#6 Groupby
grouped = df.groupby('Region')['Sales'].sum()
```

```

print("\n*****Group By*****\n")
print(grouped)

#7 Groupby and Aggregate
grouped2 = df.groupby('Region').agg({
    'Sales': 'sum',
    'Sales_with_VAT': 'mean'
})
print("\n*****Group By Aggregate*****\n")
print(grouped2)

#8 Pivot Table
pivot = pd.pivot_table(
    df,
    values='Sales',
    index='Region',
    columns='Date',
    aggfunc='sum',
    fill_value=0
)
print("\n*****Pivot Table*****\n")
print(pivot)

#9 Data Anaysis
print("\n*****Data Analysis*****\n")
print("\nMean: \n", df[['Sales', 'Sales_with_VAT']].mean())
print("\nMedian: \n\n", df[['Sales',
    'Sales_with_VAT']].median())
print("\nStandard Deviation: \n", df['Sales'].std())
print("\nVariance: \n", df['Sales'].var())
print("\nMin: \n", df[['Sales', 'Sales_with_VAT']].min())
print('\nMaximum: \n', df[['Sales',
    'Sales_with_VAT']].max())

#10 Export to excel
df.to_excel('Cleaned.xlsx', index=False)
grouped.to_excel('Groupby.xlsx')
grouped2.to_excel('Aggregate.xlsx')
pivot.to_excel('Pivot.xlsx'))

```

This example shows how to load data, create new fields, clean missing entries, and perform group summaries as well as pivot table analysis.

5.5 Matplotlib for Data Visualization

1. What Is Matplotlib?

Matplotlib is a comprehensive, open-source plotting library in Python, primarily used for creating static, interactive, and animated visualizations. Developed by John D. Hunter, it provides a MATLAB-like interface through its pyplot module and is highly customizable for professional data presentation.

It's widely used in data science, machine learning, and scientific computing to generate plots such as line charts, bar graphs, histograms, pie charts, scatter plots, and more.

2. Installation and Setup

To install Matplotlib:

```
pip install matplotlib
```

To start using it:

```
import matplotlib.pyplot as plt
```

Optionally, include %matplotlib inline in Jupyter notebooks to render charts inline.

3. Basic Components of a Matplotlib Plot

A typical Matplotlib plot consists of:

- **Figure:** The entire plotting area.
 - **Axes:** The area where data is plotted (like a subplot).
 - **Axis:** X and Y-axis objects.
 - **Labels and Titles:** Customizable text annotations.
 - **Legends:** Key for different plot elements.
 - **Ticks:** Marks on axes.
-

4. Basic Plotting Examples

a. Line Plot

```
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]
plt.plot(x, y)
plt.title("Line Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.show()
```

b. Bar Plot

```
products = ['A', 'B', 'C']
sales = [100, 150, 90]
plt.bar(products, sales)
plt.title("Product Sales")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.show()
```

c. Histogram

```
import numpy as np
ages = np.random.randint(15, 60, 100)
plt.hist(ages, bins=10, color='skyblue')
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

d. Scatter Plot

```
x = [1, 2, 3, 4, 5]
y = [2.5, 2.8, 3.6, 4.5, 5.1]
plt.scatter(x, y, color='green')
plt.title("Scatter Plot Example")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

5. Customization Options

Matplotlib allows a high level of customization:

- Line styles: `linestyle='--', color='red', marker='o'`
- Plot size: `plt.figure(figsize=(10,6))`
- Font styles and sizes
- Annotations: `plt.text()`, `plt.annotate()`
- Legends: `plt.legend()`

6. Subplots and Multiple Charts

You can create multiple plots in a single figure using `plt.subplot()`:

```
plt.subplot(1, 2, 1)
plt.plot([1,2,3], [1,4,9])
plt.title("First Plot")
```

```
plt.subplot(1, 2, 2)
plt.plot([1, 2, 3], [1, 2, 3])
plt.title("Second Plot")

plt.tight_layout()
plt.show()
```

This feature is useful for comparing datasets or showcasing multiple perspectives.

7. Matplotlib in Data Science

Matplotlib is often used in:

- **Exploratory Data Analysis (EDA)**
- **Statistical graphics**
- **Reporting and dashboards**
- **Visual storytelling in research and publications**

It integrates well with NumPy, Pandas, and other libraries like Seaborn for enriched visualization.

5.6 Data Ingestion and Exploration in python

1. What Is Data Ingestion?

Data ingestion is the process of collecting and importing data for immediate use or storage in a database. In Python, data ingestion typically involves reading data from multiple sources such as CSV files, Excel sheets, JSON files, databases, or APIs and bringing it into a DataFrame structure using libraries like **Pandas** or **SQLAlchemy**.

2. Common Data Ingestion Methods in Python

Python supports data ingestion from a wide variety of formats:

a. CSV Files

```
import pandas as pd
df = pd.read_csv('data.csv')
```

b. Excel Files

```
df = pd.read_excel('data.xlsx')
```

c. JSON Files

```
df = pd.read_json('data.json')
```

d. Databases (e.g., MySQL, PostgreSQL)

```
from sqlalchemy import create_engine
engine =
create_engine('mysql+pymysql://user:password@localhost/dbname')
df = pd.read_sql('SELECT * FROM tablename', con=engine)
```

e. Web APIs

```
import requests
url = 'https://api.example.com/data'
response = requests.get(url)
data = response.json()
df = pd.DataFrame(data)
```

3. What Is Data Exploration?

Data exploration is the initial step in data analysis that involves examining the data to understand its structure, content, and quality. This phase helps detect anomalies, missing values, patterns, and relationships.

4. Data Exploration Techniques in Python

Using Pandas, NumPy, and visualization libraries like Matplotlib or Seaborn, you can perform the following:

a. Viewing the Data

```
df.head()          # First 5 rows
df.tail()          # Last 5 rows
df.sample(5)       # Random 5 rows
```

b. Understanding Structure

```
df.shape           # Dimensions
print(df.columns)  # Column names
df.dtypes          # Data types of each column
df.info()          # Memory usage, non-null count
```

c. Summary Statistics

```
df.describe()      # Count, mean, std, min, max, etc.
```

d. Checking for Missing or Duplicate Data

```
df.isnull().sum()  # Count of missing values per column
```

```
df.duplicated().sum() # Number of duplicate rows
```

e. Value Counts and Unique Values

```
df['ColumnName'].value_counts()  
df['ColumnName'].unique()  
df['ColumnName'].nunique()
```

5. Data Visualization for Exploration

Quick visual techniques to spot trends and patterns:

a. Histogram

```
df['Column'].hist()
```

b. Boxplot

```
import seaborn as sns  
sns.boxplot(x=df['Column'])
```

c. Scatter Plot

```
sns.scatterplot(x='Feature1', y='Feature2', data=df)
```

d. Correlation Matrix

```
import matplotlib.pyplot as plt  
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')  
plt.show()
```

6. Best Practices in Data Ingestion and Exploration

- Validate file formats and encodings.
- Handle missing or inconsistent data early.
- Standardize data types for accuracy.
- Use visual tools to detect outliers.
- Automate ingestion pipelines for repeated tasks.

Q1. Create a NumPy array and perform basic operations (addition, subtraction, mean, standard deviation).

Ans:

To work with NumPy arrays:

```
import numpy as np

# Create two NumPy arrays
a = np.array([1, 2, 3, 4])
b = np.array([4, 3, 2, 1])

# Addition
add = a + b # [5, 5, 5, 5]

# Subtraction
sub = a - b # [-3, -1, 1, 3]

# Mean
mean_val = np.mean(a) # 2.5

# Standard Deviation
std_val = np.std(a) # ~1.118
```

These operations are highly optimized and used in numerical computations.

Q2. Load a dataset into a Pandas DataFrame, clean the data, and handle missing values.

Ans:

Using Pandas to load and clean data:

```
import pandas as pd

# Load dataset
df = pd.read_csv("data.csv")

# View missing values
print(df.isnull().sum())

# Fill missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Drop rows with missing values in critical columns
df.dropna(subset=['Name', 'Salary'], inplace=True)
```

```
# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(' ',
'_')
```

Pandas offers functions like `fillna()`, `dropna()`, and `isnull()` for efficient data cleaning.

Q3. Perform data aggregation and grouping on a Pandas DataFrame.

Ans:

Grouping and aggregating data is easy with Pandas:

```
# Group by 'Department' and calculate average salary
grouped = df.groupby('department')['salary'].mean()

# Multiple aggregations
agg_stats = df.groupby('department').agg({
    'salary': ['mean', 'max', 'min'],
    'age': 'median'
})
```

This is useful for summarizing and analyzing large datasets by category.

Q4. Create a line plot and a bar chart using Matplotlib with customized labels and titles.

Ans:

Using Matplotlib for basic visualizations:

```
import matplotlib.pyplot as plt

# Sample data
months = ['Jan', 'Feb', 'Mar', 'Apr']
sales = [100, 120, 90, 150]

# Line plot
plt.plot(months, sales, marker='o', color='blue')
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.grid(True)
plt.show()

# Bar chart
```

```
plt.bar(months, sales, color='green')
plt.title('Monthly Sales Bar Chart')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.show()
```

Customizations like `title()`, `xlabel()`, `ylabel()`, and `grid()` improve chart readability.

Q5. Load large datasets from various sources (e.g., CSV, JSON, databases).

Ans:

Pandas can efficiently load large datasets from multiple sources:

- **From CSV:**

```
df = pd.read_csv('large_data.csv', chunksize=100000)  #
read in chunks
```

- **From JSON:**

```
df = pd.read_json('data.json', lines=True)
```

- **From SQL Database:**

```
import sqlite3
conn = sqlite3.connect('my_database.db')
df = pd.read_sql_query('SELECT * FROM employees', conn)
```

Using `chunksize`, `indexing`, and `selective loading` helps manage memory with large data files.

Unit 6: Big Data Analysis Using Python Libraries

6.1 Introduction to Big Data and PySpark

What is Big Data?

Big Data refers to extremely large datasets that cannot be managed, processed, or analyzed using traditional data processing tools. These datasets are characterized by the **5 Vs**:

1. **Volume** – Huge amounts of data.
2. **Velocity** – Speed at which data is generated and processed.
3. **Variety** – Different types and sources of data (text, images, video, etc.).
4. **Veracity** – Uncertainty and data quality issues.
5. **Value** – Potential insights and benefits extracted from the data.

Big Data is generated from multiple sources including social media, IoT devices, sensors, logs, transactions, and more. Analyzing Big Data allows businesses and organizations to gain insights, improve decision-making, predict trends, and enhance customer experiences.

Challenges of Big Data Processing

- Traditional tools (Excel, basic SQL, or single-machine Python scripts) struggle with performance.
 - Data may not fit into memory.
 - Need for distributed processing systems that can handle scale.
-

What Is PySpark?

PySpark is the Python API for Apache Spark – an open-source, distributed computing system designed for processing and analyzing large-scale datasets.

Apache Spark supports multiple languages (Scala, Java, R, and Python) and is known for:

- High speed (up to 100x faster than Hadoop)
- In-memory computation
- Support for batch and real-time processing
- Fault-tolerance and scalability

With PySpark, Python users can interact with Spark to harness the power of parallel computing while writing code in a familiar syntax.

PySpark Setup

To get started:

```
pip install pyspark
```

Then import and create a Spark session:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("BigDataApp").getOrCreate()
```

Key PySpark Modules

1. **pyspark.sql** – Work with structured data using DataFrames.
 2. **pyspark.ml** – Machine learning library for scalable ML pipelines.
 3. **pyspark.streaming** – Real-time streaming data processing.
 4. **pyspark.sql.functions** – Built-in functions for transformations.
 5. **pyspark.rdd** – Low-level resilient distributed datasets (RDDs).
-

Applications of PySpark in Big Data

- **ETL Pipelines** – Extract, transform, and load large datasets.
- **Log Analysis** – Process web server and system logs.
- **Real-Time Analytics** – Track user behavior in real-time (e.g., e-commerce or social media).
- **Machine Learning** – Scalable model training and evaluation.
- **Data Warehousing** – Process and query massive data stored in HDFS, AWS S3, etc.

6.2 Installing and configuring PySpark

Q. What is PySpark Installation and Configuration?

Installing and configuring PySpark is the first practical step to start working with Apache Spark through Python. Proper installation ensures you can run PySpark applications locally or in a distributed environment without errors. Configuration allows you to tailor Spark's settings—such as memory allocation and number of cores—to optimize performance for your specific use case.

Step 1: Installing Java

Apache Spark runs on the Java Virtual Machine (JVM), so having Java installed is mandatory.

- **Check if Java is installed** by running in your terminal or command prompt:

```
java -version
```

- If not installed, download and install **Java Development Kit (JDK) version 8 or above** from the official Oracle website or adopt OpenJDK.
-

Step 2: Installing Apache Spark

You can install Spark in multiple ways, but the easiest for Python users is to use **PySpark via pip**, which bundles Spark binaries.

```
pip install pyspark
```

This command installs PySpark along with all necessary Spark components.

Alternatively, you can download Spark from the official Apache Spark website and set environment variables manually, but pip installation is recommended for simplicity.

Step 3: Setting Environment Variables (Optional but Recommended)

For more advanced control or standalone Spark installations, you may need to set environment variables such as:

- **SPARK_HOME** — Path to your Spark installation directory.
- **JAVA_HOME** — Path to your Java installation.
- Add Spark and Hadoop binaries to your system **PATH**.

Example for Linux/macOS:

```
export SPARK_HOME=/path/to/spark
export JAVA_HOME=/path/to/java
export PATH=$PATH:$SPARK_HOME/bin:$JAVA_HOME/bin
```

For Windows, environment variables can be set via System Properties > Environment Variables.

Step 4: Verifying the Installation

To confirm the installation, open a Python shell and try importing PySpark:

```
from pyspark.sql import SparkSession
```

```
spark =  
SparkSession.builder.appName("TestApp").getOrCreate()  
print(spark.version)
```

If this runs without errors and prints the Spark version, your setup is successful.

Step 5: Configuring SparkSession

When creating a Spark session, you can configure resources and options to optimize Spark for your hardware or cluster environment.

Example:

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("MyApp") \  
    .master("local[*]") \  
    locally # Use all available CPU cores  
    .config("spark.executor.memory", "2g") \  
    executor memory # Assign 2GB  
    .getOrCreate()
```

Q. Important Configuration Parameters

- **master:** Specifies the Spark deployment mode. `local[*]` runs Spark locally using all CPU cores.
 - **spark.executor.memory:** Amount of memory allocated per executor.
 - **spark.driver.memory:** Memory allocated for the driver program.
 - **spark.sql.shuffle.partitions:** Number of partitions for shuffle operations (default is 200).
-

Q. Troubleshooting Tips

- Make sure Java is properly installed and compatible.
- If Spark version mismatches occur, reinstall PySpark.
- Check for conflicting Python versions or virtual environments.
- On Windows, running Spark sometimes requires setting environment variables carefully.
- If you face memory issues, tweak executor and driver memory settings.

6.3 PySpark DataFrames

Q. What Is a PySpark DataFrame?

A **DataFrame** in PySpark is a distributed collection of data organized into named columns, similar to a table in a relational database or a Pandas DataFrame. It is one of the core abstractions in Apache Spark and enables scalable and efficient processing of large datasets.

PySpark DataFrames are built on top of Resilient Distributed Datasets (RDDs) but provide a higher-level API optimized for performance and ease of use. They support various data sources and allow SQL-like queries, making data manipulation intuitive.

Q. Why Use PySpark DataFrames?

- **Scalability:** Designed to handle massive datasets distributed across multiple nodes.
 - **Performance:** Uses Spark's Catalyst optimizer to plan efficient execution.
 - **Flexibility:** Supports structured and semi-structured data (CSV, JSON, Parquet, etc.).
 - **Integration:** Can be easily integrated with Spark SQL, machine learning libraries, and streaming data.
 - **Familiar Syntax:** Similar to Pandas and SQL, making it accessible for Python users.
-

Q. Creating a PySpark DataFrame

1. From Existing Data (Python List or Dictionary)

```
from pyspark.sql import SparkSession

spark =
SparkSession.builder.appName("DataFrameExample").getOrCreate()

data = [("Alice", 34), ("Bob", 45), ("Cathy", 29)]
columns = ["Name", "Age"]

df = spark.createDataFrame(data, schema=columns)
df.show()
Output:
pg
+-----+-----+
| Name|Age|
+-----+-----+
|Alice| 34|
|  Bob| 45|
```

```
|Cathy| 29|  
+-----+----+
```

2. From External Data Sources

You can create DataFrames by loading data from files like CSV, JSON, Parquet, or from databases.

Example: Loading a CSV file

```
df = spark.read.csv("data/sample.csv", header=True,  
inferSchema=True)  
df.show(5)
```

Q. Basic DataFrame Operations

- **Show data**

```
df.show() # Displays the first 20 rows by default  
df.show(5) # Displays first 5 rows
```

- **Print schema**

```
df.printSchema()
```

- **Select columns**

```
df.select("Name").show()
```

- **Filter rows**

```
df.filter(df.Age > 30).show()
```

- **Add new column**

```
from pyspark.sql.functions import col  
  
df = df.withColumn("AgePlusTen", col("Age") + 10)  
df.show()
```

- **Group and aggregate**

```
df.groupBy("Age").count().show()
```

Q. DataFrame Transformations vs Actions

- **Transformations** are lazy operations that define a computation, e.g., filter(), select(), withColumn(). They do not execute immediately but build up a logical plan.
- **Actions** trigger execution and return results, e.g., show(), collect(), count().

Q. Schema and Data Types

PySpark supports various data types including String, Integer, Float, Boolean, Date, and Timestamp.

You can define a schema explicitly using StructType and StructField for better control:

```
from pyspark.sql.types import StructType, StructField,
StringType, IntegerType

schema = StructType([
    StructField("Name", StringType(), True),
    StructField("Age", IntegerType(), True)
])

df = spark.createDataFrame(data, schema=schema)
df.printSchema()
```

Q. Common DataFrame Functions

- describe() — Provides summary statistics.
 - dropDuplicates() — Removes duplicate rows.
 - fillna() / dropna() — Handles missing data.
 - orderBy() / sort() — Sorts rows by column values.
 - join() — Joins two DataFrames on a key column.
-

Q. Example: Simple DataFrame Workflow

```
# Load data
df = spark.read.csv("data/employees.csv", header=True,
inferSchema=True)

# View data
df.show(5)

# Filter employees older than 40
df.filter(df.Age > 40).show()

# Add a new column for seniority
df = df.withColumn("Seniority", df.Age - 40)

# Aggregate average salary by department
df.groupBy("Department").avg("Salary").show()
```

6.4 Big Data Analysis with PySpark

Q. Understanding Big Data Analysis with PySpark

Big Data analysis involves processing and examining extremely large and complex datasets that traditional data tools cannot handle effectively. PySpark, the Python API for Apache Spark, offers a robust platform to perform this analysis efficiently by leveraging distributed computing across clusters.

PySpark simplifies handling Big Data by providing scalable data structures like DataFrames and RDDs (Resilient Distributed Datasets), advanced SQL capabilities, machine learning libraries, and streaming analytics—all accessible using Python.

Q. Key Features of PySpark for Big Data Analysis

- **Distributed Processing:** PySpark breaks large datasets into partitions that are processed in parallel across multiple nodes.
- **In-Memory Computation:** Speeds up iterative tasks by caching data in memory.
- **Fault Tolerance:** Automatically recovers from node failures without losing data or computation.
- **Versatile Data Support:** Handles structured, semi-structured, and unstructured data from various sources (HDFS, S3, databases, etc.).
- **Integrated Libraries:** Support for SQL, machine learning (MLlib), graph processing (GraphX), and streaming (Structured Streaming).

Q. Workflow for Big Data Analysis in PySpark

1. **Data Ingestion:** Load data from distributed storage or external sources into PySpark DataFrames or RDDs.
2. **Data Cleaning and Preprocessing:** Handle missing values, filter noise, and transform raw data into usable formats.
3. **Data Transformation:** Use DataFrame operations and SQL queries to manipulate data, create derived features, and aggregate results.
4. **Exploratory Data Analysis (EDA):** Summarize key statistics and visualize data distributions using PySpark or integration with Python visualization libraries.
5. **Model Building:** Use MLlib or integrate with other ML frameworks to train predictive models on Big Data.

6. **Evaluation and Optimization:** Assess model performance and tune hyperparameters; optimize Spark configurations for efficient resource usage.

Q. Example: Analyzing Large Dataset with PySpark

```
from pyspark.sql import SparkSession

# Initialize Spark session
spark =
SparkSession.builder.appName("BigDataAnalysis").getOrCreate
()

# Load a large dataset from CSV (for example)
df = spark.read.csv("hdfs://path/to/large_dataset.csv",
header=True, inferSchema=True)

# Show schema and preview data
df.printSchema()
df.show(5)

# Filter rows with a condition
filtered_df = df.filter(df["value"] > 1000)

# Group data by a categorical column and calculate average
agg_df = filtered_df.groupBy("category").avg("value")

# Show aggregated results
agg_df.show()

# Cache the DataFrame for faster reuse
filtered_df.cache()

# Stop Spark session after analysis
spark.stop()
```

Q. Best Practices for Big Data Analysis Using PySpark

- **Optimize Data Partitioning:** Adjust the number of partitions for parallelism based on cluster size and workload.
- **Cache Strategically:** Cache intermediate DataFrames only if reused multiple times.
- **Broadcast Variables:** Use broadcast joins for small lookup tables to improve join performance.
- **Avoid Shuffles:** Minimize wide transformations like groupBy and join where possible.

- **Use Built-in Functions:** Prefer PySpark's optimized built-in functions over user-defined functions (UDFs) for performance.
 - **Monitor and Tune:** Use Spark's web UI to monitor job execution and tune settings such as executor memory and cores.
-

Q. Real-World Applications

- **E-commerce:** Analyze user transactions and behavior to recommend products and detect fraud.
- **Healthcare:** Process large-scale patient records for predictive health analytics.
- **Finance:** Perform risk analysis and portfolio optimization on vast financial datasets.
- **Telecommunications:** Monitor network traffic and usage patterns for performance and security.
- **Social Media:** Analyze real-time streams of posts, comments, and trends.

6.5 Using SQL (DBMS tools) for querying data

Q. Introduction to SQL in Data Analysis

SQL, or Structured Query Language, is a standardized programming language designed for managing and manipulating relational databases. It remains one of the most powerful and widely used tools for querying structured data efficiently. In Big Data environments, SQL plays a critical role by allowing users to perform complex queries, aggregations, and data transformations on large datasets, often integrated with database management systems (DBMS) or distributed data platforms.

Q. Importance of SQL in Data Analysis and Big Data

- **Ease of Use:** SQL syntax is intuitive and declarative, allowing users to specify *what* they want to retrieve rather than *how* to retrieve it.
- **Data Access:** It provides a direct way to access and manipulate structured data stored in tables, views, and indexes.
- **Integration:** Modern Big Data tools, including PySpark, integrate SQL querying capabilities, bridging the gap between traditional databases and distributed data processing.
- **Versatility:** Supports filtering, sorting, joining, grouping, and advanced analytics functions.

- **Performance:** Optimized DBMS query engines execute SQL commands efficiently, often using indexes and query planners.

Q. Basic SQL Commands for Data Querying

1. **SELECT:** Retrieve specific columns or all columns from a table.

```
SELECT * FROM employees;  
SELECT name, age FROM employees WHERE age > 30;
```

2. **WHERE:** Filter rows based on conditions.

```
SELECT * FROM sales WHERE amount > 1000;
```

3. **JOIN:** Combine rows from two or more tables based on a related column.

```
SELECT orders.id, customers.name  
FROM orders  
JOIN customers ON orders.customer_id = customers.id;
```

4. **GROUP BY:** Aggregate data by one or more columns.

```
SELECT department, AVG(salary)  
FROM employees  
GROUP BY department;
```

5. **ORDER BY:** Sort query results.

```
SELECT * FROM products ORDER BY price DESC;
```

6. **INSERT, UPDATE, DELETE:** Modify data within tables.

Q. SQL Querying in Big Data Tools

Many Big Data platforms provide SQL-like interfaces to query data efficiently without writing complex code:

- **Apache Hive:** Data warehouse software built on Hadoop, supporting SQL queries over large datasets stored in HDFS.
- **Spark SQL:** Module of Apache Spark that allows executing SQL queries over DataFrames and datasets.
- **Presto and Trino:** Distributed SQL query engines that support querying data where it lives, including Hadoop, S3, and relational databases.
- **Relational Databases:** Traditional DBMS like MySQL, PostgreSQL, and SQL Server remain important for transactional and analytical workloads.

Q. Using SQL with PySpark (Spark SQL)

PySpark seamlessly integrates SQL through Spark SQL, enabling users to run SQL queries on Spark DataFrames:

- **Creating a temporary view**

```
df.createOrReplaceTempView("employee_view")
```

- **Running SQL queries**

```
result = spark.sql("SELECT department, AVG(salary) as  
avg_salary FROM employee_view GROUP BY department")  
result.show()
```

This integration combines SQL's expressiveness with Spark's distributed computing power.

Q. Advantages of Using SQL for Querying Data

- Simplifies complex data operations.
- Enables non-programmers to analyze data.
- Supports data integrity and security through role-based access.
- Provides standards-based querying that is portable across many systems.
- Enhances productivity by using familiar syntax.

6.6 Data Visualization and Reporting

Q. Introduction to Data Visualization

Data visualization is the graphical representation of data and information. By using visual elements like charts, graphs, maps, and dashboards, it helps transform complex datasets into understandable, actionable insights. Visualization makes it easier to identify patterns, trends, outliers, and relationships within the data, which might be difficult to detect from raw numbers alone.

Q. Importance of Data Visualization in Big Data Analysis

- **Simplifies Complexity:** Large datasets can be overwhelming; visualization condenses this information into digestible formats.
- **Enhances Decision-Making:** Visual summaries help stakeholders quickly grasp critical insights and make informed decisions.

- **Identifies Patterns and Trends:** Visualization highlights temporal changes, correlations, and clusters.
 - **Facilitates Communication:** Visual tools bridge gaps between data analysts and non-technical audiences.
 - **Enables Real-Time Monitoring:** Dashboards and live reports provide immediate feedback on operations or system statuses.
-

Q. Common Types of Data Visualizations

- **Bar and Column Charts:** Compare quantities across categories.
 - **Line Graphs:** Show trends over time.
 - **Pie Charts:** Display proportions and percentages.
 - **Scatter Plots:** Reveal correlations or distributions.
 - **Heatmaps:** Show intensity of data points in two dimensions.
 - **Histograms:** Depict frequency distributions.
 - **Dashboards:** Combine multiple visualizations for comprehensive reporting.
-

Q. Tools and Libraries for Data Visualization in Python and Big Data

- **Matplotlib and Seaborn:** Popular Python libraries for creating static and interactive plots.
 - **Plotly and Bokeh:** Enable dynamic, web-based visualizations.
 - **Tableau and Power BI:** Enterprise-grade platforms for interactive dashboards and reports.
 - **Integration with PySpark:** PySpark can output processed data which can then be visualized using Python libraries or third-party tools.
-

Q. Reporting in Big Data Environments

Reporting involves summarizing data analysis results into structured documents or dashboards that communicate findings effectively. Automated reporting tools can generate periodic reports, alert dashboards, and visualization summaries essential for business intelligence.

Q. Example: Visualizing PySpark Data

```
# Assuming a Spark DataFrame 'df'
pandas_df = df.toPandas() # Convert to Pandas DataFrame
for visualization
```

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.barpplot(x='category', y='sales', data=pandas_df)
plt.title('Sales by Category')
plt.show()
```

6.7 Concept of Spark SQL and Its Application

Q. What is Spark SQL?

Spark SQL is a Spark module designed for structured data processing. It provides a programming interface for working with structured and semi-structured data using SQL queries, DataFrames, and Datasets. Spark SQL seamlessly integrates relational data processing with Spark's functional programming API, allowing users to leverage SQL for big data analytics.

Q. Key Features of Spark SQL

- **Unified Data Access:** Supports various data sources like JSON, Hive tables, Parquet, ORC, and JDBC databases.
 - **DataFrames API:** Offers a domain-specific language (DSL) for querying structured data.
 - **Performance Optimization:** Uses the Catalyst optimizer to generate efficient query plans.
 - **Interoperability:** Allows mixing SQL queries and DataFrame operations within the same application.
 - **Compatibility:** Supports HiveQL and can run queries on Hive metastore tables.
-

Q. Spark SQL Architecture

Spark SQL operates through several layers:

- **Data Sources:** Input from JSON, CSV, Parquet, Hive, etc.
 - **Catalyst Optimizer:** Analyzes and optimizes SQL queries and DataFrame operations.
 - **Tungsten Execution Engine:** Efficiently executes the optimized physical plan.
 - **User Interface:** APIs for SQL, DataFrames, and Datasets in Python, Scala, Java, and R.
-

Q. Applications of Spark SQL

- **Interactive Querying:** Enables fast and flexible queries on big data.

- **ETL Pipelines:** Used to transform and prepare data for analytics.
 - **Business Intelligence:** Powers dashboards and reporting systems.
 - **Data Warehousing:** Facilitates data aggregation and integration from various sources.
 - **Machine Learning Pipelines:** Prepares features and labels using SQL transformations.
-

Q. Example: Using Spark SQL

```
from pyspark.sql import SparkSession

spark =
SparkSession.builder.appName("SparkSQLExample").getOrCreate
()

# Load data into a DataFrame
df = spark.read.json("data/people.json")

# Create temporary SQL view
df.createOrReplaceTempView("people")

# Run SQL query
results = spark.sql("SELECT name, age FROM people WHERE age
> 30")

results.show()
```

Q1. Set up a PySpark environment and create a simple PySpark application.

Ans:

To set up PySpark:

1. **Install PySpark** using pip:

```
pip install pyspark
```

2. **Create a basic PySpark script:**

```
from pyspark.sql import SparkSession

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("SimpleApp") \
    .getOrCreate()

# Create a simple DataFrame
data = [("Alice", 25), ("Bob", 30), ("Charlie", 28)]
df = spark.createDataFrame(data, ["Name", "Age"])

df.show()
spark.stop()
```

This creates a Spark session and shows a basic DataFrame.

Q2. Load a large dataset into a PySpark DataFrame and perform basic operations (filtering, grouping).

Ans:

To load and process large datasets in PySpark:

```
# Load a CSV file into DataFrame
df = spark.read.csv("bigdata.csv", header=True,
inferSchema=True)

# Filter rows where age > 30
filtered = df.filter(df['age'] > 30)

# Group by department and count
grouped = df.groupBy("department").count()

filtered.show()
grouped.show()
```

PySpark handles big datasets efficiently using distributed processing.

Q3. Handle missing data in a PySpark DataFrame and perform data cleaning.

Ans:

You can identify and handle missing data in PySpark using the na functions:

```
# Check for null values
df.select([count(when(col(c).isNull(), c)).alias(c) for c in
df.columns]).show()

# Drop rows with any nulls
clean_df = df.dropna()

# Fill missing values
filled_df = df.fillna({"salary": 0, "department": "Unknown"})
```

These methods help prepare data for reliable analysis.

Q4. Use Spark SQL to query a PySpark DataFrame and analyze the results.

Ans:

You can register a DataFrame as a SQL temporary view and run queries:

```
df.createOrReplaceTempView("employees")

# Run SQL query
result = spark.sql("SELECT department, AVG(salary) as avg_salary
FROM employees GROUP BY department")
result.show()
```

This combines the power of SQL with Spark's scalability, useful for analytical queries.

Q5. Create interactive visualizations and dashboards using Pandas with Matplotlib/Seaborn and PySpark with Plotly. Generate a report summarizing the insights from the visualizations.

Ans:

Using Pandas + Seaborn:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("sales_data.csv")
sns.barplot(x="region", y="sales", data=df)
plt.title("Sales by Region")
plt.show()
```

Using PySpark + Plotly:

```
import plotly.express as px

# Convert PySpark DataFrame to Pandas
pandas_df = df.toPandas()

# Create interactive plot
fig = px.line(pandas_df, x="date", y="revenue", title="Revenue
Over Time")
fig.show()
```

Report Summary Example:

From the visualizations:

- Region A had the highest sales performance.
- Revenue showed consistent growth over Q1 and Q2.
- The interactive dashboard allows filtering by department and time.

This insight can guide future investment decisions in marketing and operations.

Interactive dashboards help stakeholders make data-driven decisions by exploring live visual trends.